

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII PROIECTULUI DE DIPLOMĂ

Subsemnatul DORITĂ ALEXANDRU-TOMI,
legitimat cu CI seria NE nr. 162032, CNP 5030508290047
autorul lucrării SISTEM DE INTEROGARE A BAZELOR DE DATE PRIN
LIMBAJ NATURAL: PROIECTARE ȘI EVALUARE EXPERIMENTALĂ

elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul de studii TEHNOLOGIA INFORMAȚIEI organizat de către Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din Iași, sesiunea SEPTEMBRIE 2026 a anului universitar 2026, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a convențiilor internaționale privind drepturile de autor.

Data

04.03.2026

Semnătura



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI

Facultatea de Automatică și Calculatoare

Domeniul: Calculatoare și Tehnologia Informației

PROIECT DE DIPLOMĂ

**Sistem de interogare a bazelor de date
prin limbaj natural:
proiectare și evaluare experimentală**

Absolvent:

Dodita Alexandru-Tomi

Coordonator științific:

Prof. univ. dr. ing. Simona Caraiman

Iași, 2026

Rezumat

Traducerea automată a întrebărilor formulate în limbaj natural în interogări SQL (*text-to-SQL*) a devenit, odată cu modelele de limbaj de mari dimensiuni, o funcționalitate integrată în majoritatea produselor de analiză a datelor. Odată cu Model Context Protocol a devenit posibilă o abordare *agentică*: în loc să primească schema în prompt, modelul primește unelte prin care explorează autonom baza de date. Lucrarea de față verifică experimental dacă această abordare este fezabilă: dacă un model care interoghează baza prin unelte produce interogarea corectă, rata de succes obținută și costurile asociate.

Contribuția principală este un cadru de evaluare reproductibil care compară trei arhitecturi de generare SQL — un singur apel cu schema integrală în prompt, o buclă agentică peste un server Model Context Protocol și un model rulat local pe placă grafică de consum, sub un protocol în care singura variabilă care diferă este arhitectura evaluată. Metrica este acuratețea de execuție: interogarea prezisă și cea de referință sunt executate, iar seturile de rezultate sunt comparate, eliminând evaluarea eronată bazată pe sintaxa textuală. Au fost măsurate cinci modele din cloud care răspund printr-un singur apel, șapte configurații agentice și șase modele rulate local, pe două baze PostgreSQL cu 93 de întrebări punctate, plus patru rulări pe BIRD-SQL dev, un set extern de 1 534 de întrebări peste 11 baze de date.

Abordarea agentică se dovedește fezabilă, dar sub reperul cu care este comparată: configurațiile agentice ajung la 79,5–95,5 % acuratețe, față de 100 % pentru un singur apel cu schema în prompt, la un cost de 9 până la 63 de ori mai mare; capacitatea modelului nu ordonează rezultatele. Un model MoE (*Mixture-of-Experts*) de 35 de miliarde de parametri rulat local pe o placă de 12 GiB egalează cel mai bun agent din familia Claude. Pe sisteme cu resurse hardware limitate, factorul limitativ nu este dimensiunea fișierului, ci densitatea: modelul cu cel mai mare volum ocupat pe disc atinge un debit de peste șase ori mai rapid decât un model dens de dimensiune semnificativ mai mică. În final, singura dimensiune nesaturată nu este dificultatea SQL, ci ambiguitatea: la întrebările fără răspuns unic din a doua etapă, niciunul dintre cele cinci modele cu apel unic nu a solicitat clarificări, pe când accesul prin unelte inversează acest comportament.

Cuvinte cheie: text-to-SQL, acuratețe de execuție, Model Context Protocol, modele de limbaj de mari dimensiuni, inferență locală, evaluare comparativă, BIRD-SQL.

Cuprins

Rezumat	ii
1 Introducere	1
1.1 Contextul și importanța temei	1
1.2 Obiectivele lucrării	1
1.3 Metodologia de cercetare	2
1.4 Structura lucrării	2
2 Fundamentarea teoretică și soluții similare	3
2.1 Problema text-to-SQL și metrica potrivită	3
2.2 Modele de limbaj și tehnica de prompting	3
2.3 Evoluția seturilor de evaluare și problema saturării	4
2.3.1 Analiza comparativă a rezultatelor publicate	4
2.3.2 Taxonomia erorilor la frontieră	5
2.4 Legarea de schemă și abordarea agentică	5
2.5 Modele specializate și inferență locală	5
2.6 Soluții similare existente	6
2.7 Lacune identificate în stadiul actual	6
2.8 Specificații rezultate	7
3 Soluția propusă	8
3.1 Reiterarea problemei și strategia de rezolvare	8
3.2 Arhitectura sistemului de interogare	8
3.3 Cele două configurații arhitecturale	8
3.3.1 Configurația RAG — conducta proprie a sistemului	8
3.3.2 Configurația MCP — arhitectura agentică	8
3.4 Cadrul de evaluare	10
3.4.1 Structura	10
3.4.2 Mecanisme de protecție împotriva rezultatelor false	11
3.4.3 Cele trei configurații măsurate	11
3.5 Bazele de date și seturile de întrebări	11
3.5.1 Categoriile de întrebări	12
3.6 Alegerea tehnologiilor	13
4 Testarea soluției și rezultate experimentale	14
4.1 Metodologia de testare	14
4.1.1 Punerea în funcțiune	14
4.1.2 Parametrii de testare	14
4.1.3 Criterii de evaluare și punctare	14
4.2 Rezultatul de ansamblu	14
4.3 Configurația A — un singur apel	15
4.4 Configurația B — agentic	16
4.4.1 Eficiența accesului prin unelte în raport cu linia de referință	16
4.4.2 Costul	17
4.5 Configurația C — modele rulate local	17
4.5.1 Acuratețea	17
4.5.2 Constrângerea de performanță: modele dense versus MoE	18

4.5.3	Rezultate experimentale secundare	19
4.6	Analiza factorilor de dificultate	19
4.7	Ambiguitatea, singura dimensiune nesaturată	20
4.8	Validarea externă pe BIRD-SQL	21
4.8.1	Rezultate	22
4.8.2	Eliminarea saturației de evaluare prin setul BIRD	22
4.8.3	Comparația de performanță: modelul pro versus modelul flash	22
4.8.4	Impactul dicționarului de date asupra acurateței	23
4.8.5	Dispersia între baze de date	24
4.8.6	Validarea manuală a eșecurilor de execuție	24
4.8.7	Note de metodă	25
4.9	Amenințări la adresa validității și limitări	25
4.9.1	Anomalii identificate în cadrul de evaluare și remedierea acestora	25
4.9.2	Acoperire	26
4.9.3	Considerații privind evaluarea conduitei RAG proprii	26
4.9.4	Limite cunoscute ale măsurătorii	26
5	Concluzii	27
5.1	Contribuții personale	27
5.2	Analiza rezultatelor	27
5.3	Direcții de dezvoltare	28
	Bibliografie	29
	Anexe	30
A	Modelele măsurate	31
B	Reproducerea rezultatelor	32
	Structura seturilor de întrebări	32

1. Introducere

1.1 Contextul și importanța temei

Datele operaționale ale unei organizații sunt stocate aproape universal în baze de date relaționale, iar accesul la ele trece prin SQL. Această dependență creează o barieră tehnică bine documentată: persoanele care au nevoie de răspunsuri (analști de business, manageri, personal operativ) nu sunt, de regulă, aceleași cu persoanele care știu să scrie interogări. Rezultatul practic este un decalaj de câteva zile între întrebare și răspuns, cu un intermediar tehnic pe traseu.

Problema traducerii automate din limbaj natural în SQL este cunoscută în literatura de specialitate sub denumirea *text-to-SQL* și a fost studiată sistematic de la introducerea setului de date Spider [1]. Modelele de limbaj de mari dimensiuni au schimbat radical situația: pe seturile clasice de evaluare, scorurile au trecut în câțiva ani de la aproximativ 70 % la valori apropiate de maximul posibil. Această creștere creează însă o problemă de măsurare: când aproape toate sistemele răspund corect la aproape toate întrebările, scorul nu mai poate arăta care sistem este mai bun. Setul de evaluare devine saturat, iar acesta este punctul de plecare al lucrării de față.

A apărut, în paralel, o a doua abordare posibilă. În loc să primească schema bazei în prompt, modelul primește *unelte*: poate lista tabelele, poate descrie o tabelă, poate rula interogări exploratorii și poate decide autonom când dispune de informațiile necesare generării răspunsului final. Model Context Protocol (MCP) [9] a standardizat acest tipar, iar adoptarea lui a fost rapidă: la un an de la lansare, protocolul raporta peste 97 de milioane de descărcări lunare ale bibliotecilor sale și aproximativ 10 000 de servere active, cu suport nativ în principalele produse de asistență bazate pe modele de limbaj. În decembrie 2025 protocolul a trecut sub guvernanta Agentic AI Foundation, fundație a Linux Foundation susținută de principalii furnizori de modele și de infrastructură cloud [19]. Argumentul este intuitiv: un model care poate inspecta baza de date ar trebui să scrie interogări mai bune decât unul care primește doar o descriere textuală a ei.

Argumentul este însă o predicție, nu un rezultat măsurat, iar lucrarea de față îl tratează ca atare. Problema fundamentală nu este dacă un sistem text-to-SQL este realizabil tehnic (arhitectura unui astfel de sistem fiind detaliată în capitolul 3), ci dacă accesul agentic la baza de date este o abordare *fezabilă*: dacă produce interogări corecte, cu ce rată, la ce cost și cu ce moduri de eșec. Celelalte două configurații măsurate (un singur apel cu schema în prompt și un model rulat local) servesc drept repere pentru interpretarea acestor cifre, nu ca alternative aflate în competiție cu ea.

1.2 Obiectivele lucrării

Obiectivul general este proiectarea unui cadru de evaluare reproductibil pentru sisteme text-to-SQL și utilizarea lui pentru a stabili, sub un protocol controlat, dacă generarea SQL prin acces agentic la baza de date este fezabilă, raportat la două repere măsurate în aceleași condiții.

Obiectivele specifice sunt:

1. **Definirea unui protocol de măsurare valid.** Metrica trebuie să fie acuratețea de execuție (compararea seturilor de rezultate obținute prin rularea efectivă a interogărilor) și nu asemănarea textuală cu interogarea de referință, care penalizează formulări corecte, dar diferite.
2. **Izolarea variabilei studiate.** Un model rulat local și unul din cloud primesc același prompt, același contract de ieșire și același mediu de execuție, astfel încât diferența de acuratețe dintre ele să fie atribuibilă modelului.
3. **Măsurarea accesului agentic la baza de date** (o buclă peste un server MCP) pe seturi de întrebări de dificultate controlată și pe scheme de dimensiuni diferite, împreună cu două repere obținute în aceleași condiții: un singur apel cu schema integrală în prompt și un model rulat local.
4. **Validarea externă.** Rezultatele obținute pe întrebări formulate de autorul sistemului trebuie confirmate pe un set de întrebări elaborat independent, cu metrică publicată și clasament public —

BIRD-SQL [2].

5. **Evaluarea fezabilității rulării locale** a modelelor de limbaj pe hardware de consum, cu identificarea factorului care limitează efectiv performanța.
6. **Identificarea dimensiunii nesaturate:** a acelei capacități pe care niciun sistem măsurat nu o are, și care indică direcția de dezvoltare utilă.

1.3 Metodologia de cercetare

Lucrarea urmează o metodologie experimentală. Etapele au fost:

1. **Studiul literaturii** pentru identificarea claselor de dificultate reale în text-to-SQL (Spider 2.0 [3], BIRD [2], BEAVER [4]) și a problemelor de validitate cunoscute ale seturilor de evaluare din acest domeniu [5].
2. **Construirea seturilor de întrebări** pe categorii derivate din taxonomia de erori publicată în literatură, incluzând o categorie de întrebări deliberat ambigue.
3. **Implementarea cadrului de evaluare** cu execuție reală a ambelor interogări, amprentare criptografică a setului de întrebări și înregistrarea per-întrebare a SQL-ului, a latenței, a consumului de tokeni și a costului.
4. **Rularea celor trei configurații** și analiza per-categorie a eșecurilor, prin reexecutarea manuală a interogărilor greșite.
5. **Validarea externă** pe BIRD-SQL dev, cu toate cele 1 534 de întrebări, fără eșantionare.

1.4 Structura lucrării

Capitolul 2 prezintă fundamentarea teoretică, soluțiile existente și lacunele identificate în stadiul actual. Capitolul 3 descrie soluția propusă: sistemul de interogare, cele două configurații arhitecturale și, în principal, cadrul de evaluare care constituie contribuția principală. Capitolul 4 prezintă protocolul de măsurare și rezultatele experimentale, alături de analiza aprofundată a concluziilor ce decurg din acestea. Capitolul 5 sintetizează contribuțiile și direcțiile de dezvoltare.

2. Fundamentarea teoretică și soluții similare

2.1 Problema text-to-SQL și metrica potrivită

Formal, un sistem text-to-SQL primește o întrebare în limbaj natural q și o schemă S și produce o interogare $\tilde{y}$ care, executată peste baza de date D , trebuie să returneze răspunsul corect. Există două familii de metrice pentru a decide dacă $\tilde{y}$ este corectă.

Potrivirea exactă compară $\tilde{y}$ cu interogarea de referință y la nivel de text sau de arbore sintactic. Este ieftină, dar sistematic pesimistă: două interogări diferite pot fi ambele corecte, iar un JOIN scris ca subinterogare este penalizat fără motiv.

Acuratețea de execuție rulează ambele interogări și compară seturile de rezultate:

$$\text{EX}(\tilde{y}, y) = \begin{cases} 1, & \text{dacă } \text{exec}(D, \tilde{y}) \equiv \text{exec}(D, y), \\ 0, & \text{altfel,} \end{cases}$$

unde $\equiv$ este egalitatea de multimulțimi de tupluri, ignorând numele coloanelor și (dacă întrebarea nu cere explicit o ordine) ordinea rândurilor. Aceasta este metrica adoptată în lucrarea de față și metrica oficială a BIRD [2].

Acuratețea de execuție are însă un defect cunoscut: este o condiție necesară, nu suficientă, pentru corectitudine. O interogare care produce în mod accidental același set de rânduri este punctată drept corectă, iar o analiză sistematică a erorilor semantice din traduceri text-to-SQL [7] arată că astfel de cazuri nu sunt rare: interogări care folosesc un JOIN greșit, dar produc același rezultat pe datele existente, sau care omit o condiție care se dovedește vidă în instanța curentă. Lucrarea de față nu rezolvă această limitare: o declară explicit în secțiunea 4.9 și o compensează parțial prin inspecția manuală a eșecurilor (vezi secțiunile 4.6 și 4.8).

2.2 Modele de limbaj și tehnica de prompting

Toate sistemele măsurate în această lucrare sunt construite peste modele de limbaj cu arhitectură *transformer* [20], în care mecanismul de atenție permite fiecărei poziții din secvență să pondereze toate celelalte poziții. Consecința relevantă aici este că întreaga schemă a bazei de date, oricât de lungă, este disponibilă simultan modelului atunci când acesta generează interogarea: ceea ce face plauzibilă, încă de la început, abordarea cu un singur apel.

Modul în care este formulată cererea determină însă o parte substanțială din rezultat. Trei decizii de prompting sunt fixate identic pentru toate configurațiile comparabile din această lucrare, cu scopul de a elimina variabilele parazite:

- **Zero-shot, fără exemple rezolvate.** Promptul conține schema, domeniul, dialectul și instrucțiunea de a returna doar interogarea. Introducerea unor exemple ar fi îmbunătățit probabil scorurile, dar ar fi introdus o a doua variabilă: alegerea exemplilor.
- **Temperatură zero.** Eșantionarea stocastică ar fi transformat fiecare celulă într-o distribuție care necesită rulări repetate pentru a fi estimată; la temperatură zero, o rulare este aproape deterministă.
- **Contract de ieșire explicit.** Modelului i se cere un singur bloc de cod. Modul în care este extras acel bloc din răspuns s-a dovedit, la rândul lui, o sursă reală de eroare de măsurare (vezi secțiunea 4.9).

O a patra decizie privește *cunoașterea externă*. BIRD atașează fiecărei întrebări un câmp de context fără de care întrebarea nu este rezolvabilă; acest câmp este inclus în prompt, fiindcă face parte din definiția sarcinii, nu din soluție.

2.3 Evoluția seturilor de evaluare și problema saturării

Spider [1] a introdus evaluarea pe scheme nevăzute în antrenare și a fost referința timp de câțiva ani. Sistemele moderne au ajuns însă să obțină pe el scoruri de peste 90 %, foarte apropiate între ele, ceea ce are două consecințe distincte.

Prima este pierderea puterii de discriminare: când sistemele comparate obțin scoruri aflate la câteva zecimi unul de altul, diferențele dintre ele intră sub pragul de zgomot al măsurătorii, iar ierarhia rezultată nu mai indică în mod concludent care sistem este superior.

A doua este că scorul mare nu se transferă la sarcini reale. Spider 2.0 [3] a fost conceput pornind de la această motivație, pe fluxuri de lucru specifice mediului de întreprindere, și coboară sistemele de vârf la 21,3 %. Un scor de 90 % pe Spider descrie, prin urmare, performanța pe sarcinile specifice din Spider, nu capacitatea generală de a interoga o bază de date de producție.

BIRD [2] a răspuns prin baze de date extinse, conținând date reale și eterogene („dirty data”), și prin introducerea noțiunii de *cunoaștere externă*: un câmp de context fără de care întrebarea nu este rezolvabilă. Concluzia principală a autorilor BIRD este că plafonul nu este dat de sintaxa SQL, ci de *ancorarea în date*: potrivirea unei valori din întrebare cu reprezentarea ei efectivă în tabelă.

Spider 2.0 [3] a mutat evaluarea în context de întreprindere, cu scheme de sute de tabele și fluxuri de lucru pe mai mulți pași, unde cele mai bune sisteme obțin 21,3 %. Taxonomia de erori publicată acolo (în special utilizarea greșită a dialectului și ancorarea temporală a cohortelor) a fost folosită direct la proiectarea categoriilor de întrebări din prezenta lucrare.

BEAVER [4] a arătat că schemele reale de întreprindere, extrase din jurnale de interogări, sunt substanțial mai dificile decât cele sintetice, iar o analiză recentă [5] a documentat probleme sistematice de validitate în seturile de evaluare text-to-SQL: interogări de referință greșite, întrebări ambigue punctate binar și scoruri care nu se reproduc. Această din urmă lucrare a determinat direct structura secțiunii 4.9.

2.3.1 Analiza comparativă a rezultatelor publicate

Tabelul 2.1 rezumă traiectoria domeniului. A doua coloană indică cea mai ridicată acuratețe de execuție raportată public pe fiecare set: cu cât este mai aproape de 100 %, cu atât capacitatea de discriminare a setului scade. Spider, atingând 91 %, prezintă un grad ridicat de saturație, în timp ce Spider 2.0, la 21,3 %, păstrează o capacitate puternică de discriminare.

Tabela 2.1: Seturile de evaluare majore pentru text-to-SQL.

Set de evaluare	Acuratețe max- imă publicată	Stare	Ce măsoară efectiv
Spider [1]	≈91 %	saturat	Generalizare la scheme nevăzute; SQL de complexitate moderată
BIRD [2]	81,95 % ¹ (uman: 92,96 %)	activ, dar staționar	Ancorarea în date: potrivirea valorilor și a coloanelor slab documentate
Spider 2.0 [3]	21,3 %	departe de plafon	Forma interogării în context de întreprindere; SQL de referință de 144,5 tokeni în medie, față de 30,9 la BIRD
BEAVER [4]	—	recent	Scheme reale, extrase din jurnale de interogări de producție

Comparația arată că diferența de dificultate dintre BIRD și Spider 2.0 nu provine, în principal, din dimensiunea schemei, ci din *forma* interogării cerute: SQL-ul de referință al Spider 2.0 este de aproape cinci ori mai lung. Aceasta este observația care a determinat structura categoriilor de întrebări din capitolul 3, categoriile adăugate în etapa a doua (*fereastră, structură, măsură, temporal*) vizează forma interogării, nu numărul de tabele.

2.3.2 Taxonomia erorilor la frontieră

Analiza a 300 de eșecuri publicată odată cu Spider 2.0 [3] oferă cea mai bună clasificare disponibilă a ceea ce rămâne nerezolvat (tabelul 2.2):

Tabela 2.2: Distribuția eșecurilor la frontieră, după Spider 2.0 [3].

Clasă de eșec	Pondere
Analiză de date eronată	35,5 %
— din care: planificare complexă a interogării	17,7 %
— din care: utilizarea greșită a dialectului sau a funcțiilor	10,3 %
— din care: calcul avansat	7,5 %
Legare greșită de schemă	27,6 %
Erori de îmbinare (JOIN)	8,3 %

Ordinea este semnificativă. *Legarea de schemă*, exact ceea ce testează o schemă cu 68 de tabele și exact ceea ce urmărește să îmbunătățească o conductă de recuperare — reprezintă 27,6 %. *Planificarea și calculul* împreună reprezintă 35,5 %. Cea mai mare subclasă unică, planificarea complexă, descrie un model care emite o interogare *structural mai simplă decât cere sarcina*: mai puține niveluri de agregare decât are nevoie măsura cerută, o etapă CTE lipsă, o subinterogare colapsată.

Autorii BIRD ajung, pe cont propriu, la o concluzie complementară: obstacolul dominant pe bazele lor de date nu îl constituie construcțiile SQL exotice, ci ancorarea în schemă și în valori [2]. Cele două taxonomii nu se contrazic: descriu regimuri diferite. Pe o schemă restrânsă și bine documentată, ancorarea în date devine trivială, iar singura axă rămasă este complexitatea structurală a interogării, ceea ce implică faptul că **un set de întrebări care nu solicită structuri SQL complexe are o putere de discriminare nulă** pe o astfel de schemă. Aceasta constituie justificarea directă a categoriilor adăugate în capitolul 3 și fundamentul analizei din secțiunea 4.6.

2.4 Legarea de schemă și abordarea agentică

Pentru scheme mari, transmiterea integrală a schemei în prompt devine costisitoare, ceea ce a motivat *legarea de schemă* (*schema linking*): selectarea prealabilă a tabelelor relevante, tipic prin recuperare augmentată (RAG). Un rezultat important [6] arată însă că, pentru modelele moderne și scheme de dimensiune mică sau medie, câștigul acestei etape este marginal, iar riscul de a elimina o tabelă necesară este real.

Abordarea agentică extinde această paradigmă: modelul nu primește nici schema, nici tabelele preselectate, ci un set de unelte. Model Context Protocol [9] standardizează expunerea acestor unelte, iar un server MCP pentru PostgreSQL oferă tipic `list_tables()`, `describe_table()`, `get_schema()` și `run_query()`. Modelul rulează o buclă autonomă până când decide că poate răspunde.

Argumentul în favoarea acestei arhitecturi este că modelul poate inspecta valorile reale, având astfel potențialul de a preveni eșecurile de ancorare pe care BIRD le identifică drept plafon. **Secțiunea 4.4 măsoară dacă această predicție se confirmă experimental și arată că ipoteza este infirmată.**

2.5 Modele specializate și inferență locală

Alături de modelele generaliste din cloud există modele specializate pe text-to-SQL. Arctic-Text2SQL-R1 [8] este reprezentativ: un model de 7 miliarde de parametri antrenat prin învățare cu întărire folosind corectitudinea execuției ca recompensă, raportat în fruntea clasamentului BIRD la categoria sa de dimensiune.

Rularea locală a acestor modele pe hardware de consum ridică o problemă distinctă. Un model cuantizat este stocat într-un fișier GGUF a cărui dimensiune determină dacă poate fi încărcat integral în memoria plăcii grafice (VRAM). Când capacitatea VRAM este depășită, straturile neuronale rămase

sunt transferate în memoria RAM a sistemului și vehiculate prin magistrala PCIe. Arhitecturile cu activare parțială (*Mixture-of-Experts*, MoE) schimbă fundamental această dinamică computațională: dintr-un model de 35 de miliarde de parametri, doar aproximativ 3 miliarde sunt active pentru fiecare token generat, ceea ce permite păstrarea experților inactivi în memoria sistemului cu o penalizare minimă de latență. Secțiunea 4.5 măsoară acest efect și arată că este dominant.

Cuantizarea (reprezentarea greutăților pe mai puțini biți decât cei 16 ai antrenării) este mecanismul care face posibilă rularea locală. Ea este de obicei prezentată ca un compromis între dimensiune și calitate, cu presupunerea implicită că mai mulți biți înseamnă răspunsuri mai bune. Secțiunea 4.5 măsoară același model la două niveluri de cuantizare și arată că presupunerea nu se verifică în cazul studiat. Din acest motiv, fiecare model local este identificat în această lucrare prin *fișierul exact* rulat (anexa A): cuantizarea este parte a măsurătorii.

Serverul de inferență folosit este llama.cpp [23], care expune o interfață compatibilă cu API-ul OpenAI. Acest detaliu are o consecință metodologică importantă: permite variante locale și cele din cloud să folosească exact același cod client, deci să difere printr-o singură variabilă.

2.6 Soluții similare existente

Tabelul 2.3 sintetizează principalele clase de soluții existente în literatura de specialitate și în industrie.

Tabela 2.3: Clase de soluții existente pentru interogarea în limbaj natural.

Clasă	Exemple și mod de funcționare	Limitare relevantă
Produse comerciale integrate	Asistenți de tip „întreabă-ți datele” din suitele de <i>business intelligence</i> ; model generalist peste un strat semantic proprietar	Nu publică metrice de acuratețe; nu pot fi reproduse sau auditate
Servere MCP pentru baze de date	Expun schema și execuția ca unelte, lasă modelul să exploreze	Evalueate între ele [10], dar nu pe generarea de SQL raportată la un reper măsurat pe aceleași întrebări
Modele specializate	Arctic-Text2SQL-R1 [8], antrenate cu recompensă pe execuție	Scorul de clasament este legat de dialectul de antrenare (vezi secțiunea 4.5)
Cadre academice	Conducte cu legare de schemă, descompunere și auto-reparare [11]	Evalueate pe seturi în mare parte saturate; fără o configurație agentică în comparație

2.7 Lacune identificate în stadiul actual

Din analiza de mai sus rezultă patru lacune, care structurează contribuția acestei lucrări:

- Evaluările existente ale serverelor MCP nu acoperă acest scenariu.** MCPBench [10] compară servere MCP între ele și cu apelurile de funcții, pe sarcini de căutare web și de căutare în baze de date, iar rezultatele arată diferențe mari între implementări. Nu se măsoară însă generarea de SQL peste o schemă completă, cu agentul de programare folosit ca atare, și nici raportarea la un reper obținut pe aceleași întrebări.
- Reperele publicate nu acoperă configurația agentică.** Comparațiile sistematice existente [11] raportează varianta cu schema integrală alături de conductele cu legare de schemă, dar configurația agentică, în care modelul explorează autonom baza de date, nu apare în aceste comparații.
- Costul este raportat pentru prompturi, nu pentru bucle agentice.** Lucrările care optimizează costul [11] măsoară numărul de tokeni ai unui singur prompt. Într-o buclă agentică, contextul este retrimis la fiecare iterație, iar acest cost cumulat nu apare în comparațiile disponibile.

4. **Ambiguitatea este studiată separat de acuratețe.** Există seturi dedicate: AmbiQT [12] conține peste 3 000 de întrebări cu două interpretări SQL valide, iar PRACTIQ [13] adaugă întrebări la care răspunsul corect este o cerere de clarificare. Aceste seturi evaluează însă modele specializate pe sarcina de dezambiguizare; seturile generale (Spider, BIRD, Spider 2.0) presupun un răspuns unic per întrebare, așa că un sistem nu este măsurat simultan pe acuratețe și pe comportamentul în fața unei întrebări insuficient specificate.

2.8 Specificații rezultate

Cadrul de evaluare proiectat în capitolul 3 trebuie, prin urmare, să îndeplinească următoarele condiții:

- să puncteze prin execuție reală, comparând seturi de rezultate;
- să permită schimbarea unei singure variabile între două rulări;
- să înregistreze cost, latență și consum de tokeni împreună cu acuratețea, în același fișier;
- să conțină întrebări ambigue, punctate pe criteriul cererii de clarificare;
- să detecteze automat rulările învechite, prin amprentarea setului de întrebări;
- să fie validabil extern, pe un set de întrebări scris de terți.

3. Soluția propusă

3.1 Reiterarea problemei și strategia de rezolvare

Problema are două componente. Componenta de construcție (un sistem care primește o întrebare în limbaj natural și returnează date și rapoarte) este rezolvată printr-o arhitectură de microservicii descrisă succint în secțiunea 3.2. Componenta de măsurare (evaluarea comparativă a arhitecturilor de generare SQL) constituie contribuția principală a lucrării și formează obiectul restului acestui capitol, precum și al întregului capitol 4.

Strategia de măsurare se reduce la un principiu de proiectare experimentală: *între două rulări comparate trebuie să difere o singură variabilă experimentală*. Structura descrisă în secțiunea 3.4 reprezintă o aplicare directă a acestui principiu.

3.2 Arhitectura sistemului de interogare

Sistemul este organizat ca șapte servicii containerizate. Interfața web (React cu TypeScript) comunică prin Server-Sent Events cu serviciul RAG, care realizează generarea SQL și orchestrarea răspunsului. Un serviciu de persistență (FastAPI) păstrează conversațiile și metadatele SQL asociate, iar un back-end Rust deservește accesul la baza de date demonstrativă. Două instanțe PostgreSQL separă datele aplicației de datele interogate.

Elementul relevant pentru restul lucrării este că accesul la datele interogate se face *exclusiv* printr-un rol PostgreSQL cu drepturi de citire, cu plafon de rânduri și timp maxim de execuție per instrucțiune. Aceeași constrângere este impusă și în cadrul de evaluare, astfel încât măsurătorile să fie făcute în condițiile în care sistemul rulează efectiv.

3.3 Cele două configurații arhitecturale

3.3.1 Configurația RAG — conducta proprie a sistemului

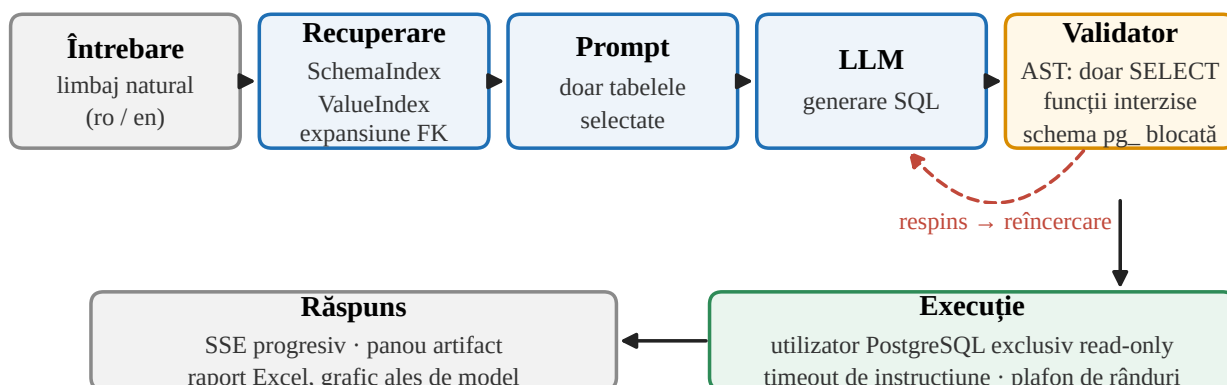
Conducta proprie (figura 3.1) nu transmite modelului schema integrală. O etapă de recuperare selectează tabelele candidate prin două indexuri complementare: un index de schemă, care combină un scor lexical cu similaritate cosinus peste înglobări ale descrierilor de tabele, și un index de valori, care leagă literalii din întrebare de coloanele în care apar efectiv. Selecția este apoi extinsă pe relațiile de cheie străină, pentru a evita cazul în care o tabelă de legătură necesară lipsește din prompt.

SQL-ul generat trece printr-un validator care operează pe arborele sintactic, nu pe text: sunt respinse toate nodurile care modifică date, schemele de sistem și un set de funcții cu efecte laterale. O interogare respinsă declanșează o reîncercare cu mesajul de eroare atașat. Abia apoi urmează execuția și generarea răspunsului, flux progresiv prin SSE, panou de tip *artifact* cu grafice, și, la cerere, raport Excel al cărui tip de grafic este ales de model.

3.3.2 Configurația MCP — arhitectura agentică

Configurația agentică (figura 3.2) inversează raportul: modelul nu primește nicio informație despre schemă în prompt, ci patru unelte expuse printr-un server MCP peste transport stdio. Serverul este un strat subțire de înregistrare a uneltelor peste un modul de acces la date fără dependențe MCP, ceea ce permite testarea logicii de bază independent de protocol.

Bucula de decizie aparține integral modelului: acesta alege ce unelte apelează, ordinea acestora și momentul în care dispune de informațiile necesare generării răspunsului. Consecința de cost este structurală: la fiecare iterație întregul context acumulat este retrimis, peste promptul de sistem al *har-ness*-ului. Media măsurată este între aproximativ 65 000 și 162 000 de tokeni de prompt per întrebare, față de 1 511 pentru un singur apel.



Configurație măsurată separat de A–C. Rândul său nu apare în capitolul 4; motivul este dat în secțiunea 4.9.

Figura 3.1: Configurația RAG: conducta proprie a sistemului. Recuperarea înlocuiește schema integrală cu tabelele selectate, iar validarea pe arbore sintactic precedă orice execuție.

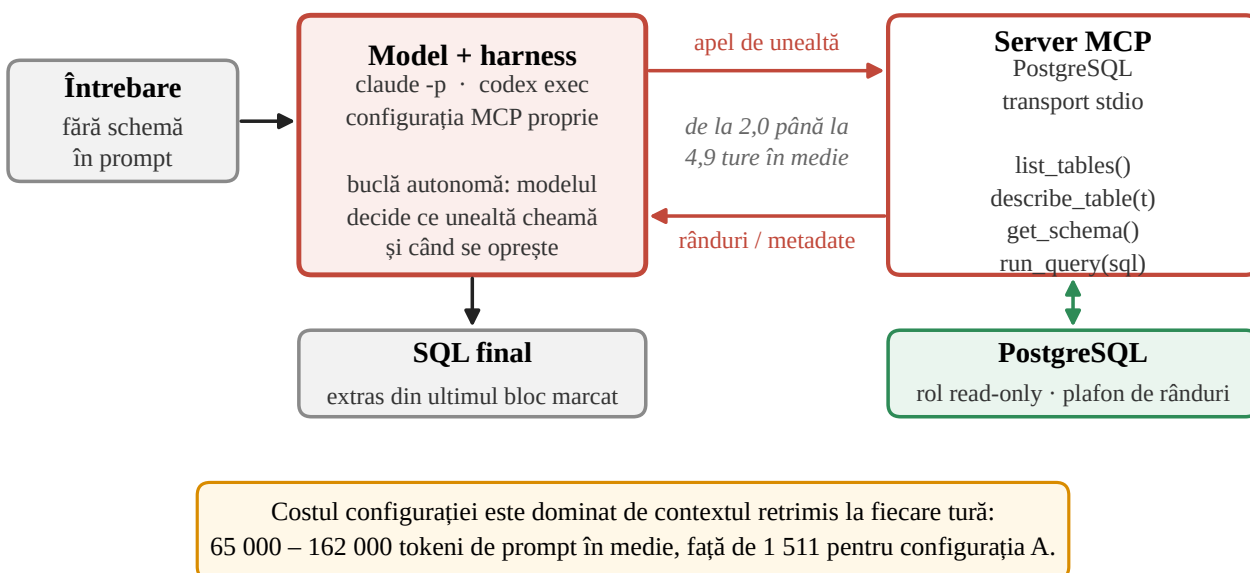
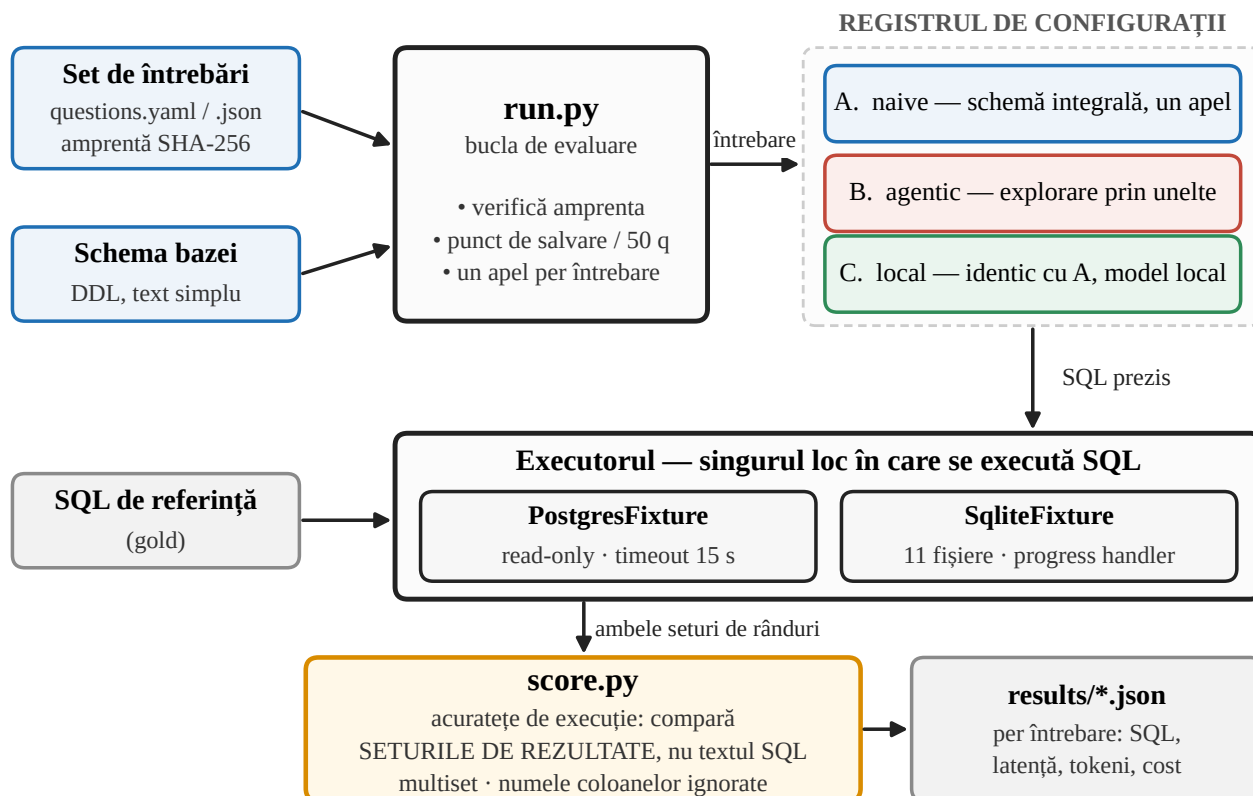


Figura 3.2: Configurația agentică peste Model Context Protocol. Ceea ce se măsoară este agentul așa cum este livrat, cu promptul său de sistem implicit.

3.4 Cadrul de evaluare

3.4.1 Structura

Cadrul de evaluare (figura 3.3) este organizat în jurul a trei abstracții, fiecare izolând o sursă de eroare care, altfel, s-ar propaga în rezultate.



Singura variabilă care diferă între configurațiile A și C este modelul care răspunde: prompt, contract de ieșire și mediu de execuție sunt identice.

Figura 3.3: Arhitectura cadrului de evaluare. Configurațiile diferă doar prin modul în care se obține SQL-ul; execuția și punctarea sunt comune și identice pentru toate.

Registrul de configurații. O configurație este o funcție cu semnătură unică: primește o întrebare și returnează SQL-ul prezis împreună cu latența, consumul de tokeni și eventualele erori. Sub această semnătură intră deopotrivă un apel HTTP către un model din cloud, un subproces care rulează un agent de linie de comandă și o cerere către un server local llama.cpp. Bucula de evaluare este complet agnostică față de mecanismul concret de execuție utilizat.

Executorul de interogări. Este singurul loc în care se execută SQL, atât cel de referință, cât și cel prezis. Existența lui este ceea ce permite cadrului de evaluare să lucreze cu două motoare diferite: PostgreSQL pentru primele două baze, SQLite pentru BIRD. Diferența nu este cosmetică. PostgreSQL oferă `statement_timeout`; SQLite nu are echivalent, așa că limita de timp este impusă printr-un *progress handler* apelat la fiecare 10 000 de instrucțiuni ale mașinii virtuale, care întrerupe execuția când termenul a expirat. De asemenea, în cazul BIRD baza de date corectă este o proprietate a *întrebării*, nu a rulării, ceea ce a impus mutarea alegerii conexiunii în interiorul executorului.

Punctarea. Modulul de punctare primește două seturi de rânduri și returnează o valoare booleană. Nu vede niciodată textul SQL. Această separare este cea care face imposibilă, prin construcție, o punctare bazată pe asemănare textuală.

3.4.2 Mecanisme de protecție împotriva rezultatelor false

Trei mecanisme au fost adăugate special pentru a împiedica producerea unor cifre plauzibile, dar greșite.

Amprentarea setului de întrebări. Fiecare fișier de rezultate conține amprenta setului de întrebări față de care a fost punctat. La generarea tabelelor, un fișier cu amprentă diferită de cea curentă este semnalat ca învechit și exclus, în loc să fie mediat în tăcere. Mecanismul a fost verificat în practică: la generarea figurilor din capitolul 4 a fost interceptată o rulare de 26 de întrebări din prima etapă, aflată în același director cu rularea curentă de 44 de întrebări a aceluiași model.

Separarea erorilor de referință de erorile de model. Interogarea de referință și cea prezisă sunt executate separat, iar eșecurile sunt înregistrate în câmpuri distincte. O interogare de referință care nu rulează nu este contabilizată ca răspuns greșit al modelului.

Refuzul referințelor triviale. Dacă o interogare de referință returnează zero rânduri, rularea este oprită cu eroare. O referință vidă ar fi validată de orice interogare care produce un rezultat vid, inclusiv de una incorectă, introducând scoruri artificiale.

3.4.3 Cele trei configurații măsurate

Tabelul 3.1 prezintă definiția operațională a fiecăreia dintre cele trei configurații evaluate.

Tabela 3.1: Definiția operațională a celor trei configurații.

Configurație	Ce primește modelul	Cine conduce bucla
A — naiv	Schema integrală ca text, un singur apel, instrucțiunea de a returna doar SQL	Procedură sincronă; apel unic
B — agentic	Fără schemă inițială; explorare prin unelte MCP până la convergență	Un agent de linie de comandă (claude -p [15], codex exec [16]) sau configurația MCP proprie
C — local	Prompt și contract <i>identice</i> cu A; diferă doar modelul	Procedură sincronă; apel unic prin llama.cpp pe GPU local

Identitatea dintre A și C este deliberată și este ceea ce face comparația interpretabilă: un rând local și unul din cloud diferă printr-o singură variabilă.

3.5 Baze de date și seturile de întrebări

Măsurătorile folosesc trei baze de date de dimensiuni și proveniențe diferite, alese astfel încât rezultatul să nu depindă de o singură schemă. Tabelul 3.2 sintetizează caracteristicile acestor baze de date și numărul de întrebări asociate.

Tabela 3.2: Cele trei baze de date de evaluare și seturile de întrebări asociate. Dimensiunea promptului este numărul de caractere al schemei transmise modelului în configurația A, măsurat din fișierele de rezultate.

Baza	Tabele	Coloane	Rânduri	Schemă (car.)	Întrebări
car_rental	9	≈70	8 026	5 951	51: 44 punctate + 7 ambigue
adventureworks	68	455	≈761 000	37 019	56: 49 punctate + 7 ambigue
BIRD-SQL dev	75	798	3 932 735	variabilă	1 534, toate punctate

BIRD-SQL dev însumează 11 baze SQLite distincte, de la 3 la 13 tabele fiecare, 1,49 GB pe disc.

car_rental este baza demonstrativă a sistemului: mică, cu schemă curată, folosită pentru a izola dificultatea SQL de dificultatea schemei. adventureworks este baza de date OLTP demonstrativă publicată de Microsoft [22], cu 68 de tabele de bază, 455 de coloane, 91 de chei străine și cinci scheme; o schemă de 37 019 caractere testează dacă avantajul configurației A rezistă când promptul crește de șase ori. BIRD-SQL dev [2] introduce un element absent din primele două baze: un set de întrebări elaborate independent de autorul lucrării. Setul este construit din două artefacte distincte, care nu sunt distribuite împreună: revizia de întrebări și interogări de referință [21] și pachetul celor 11 baze SQLite; asocierea lor, împreună cu validarea fiecărei interogări de referință, este făcută de un script dedicat (anexa B).

3.5.1 Categoriile de întrebări

Cele două seturi proprii sunt construite pe categorii derivate din taxonomia de erori din Spider 2.0 [3] și din analiza de eșecuri a BIRD [2]. Distribuția este dată în tabelul 3.3.

Tabela 3.3: Distribuția întrebărilor pe categorii. Grupurile marcate cu † au fost adăugate în etapa a doua, după ce categoriile inițiale au încetat să diferențieze sistemele.

Grup	Ce conține	car_rental	advent.
simple	selecție și filtrare pe o tabelă	6	5
agregare	GROUP BY, funcții de agregare	9	8
join	îmbinări pe două sau mai multe tabele	5	10
subinterogare	subinterogări corelate și necorelate	6	7
fereastră†	funcții fereastră, cadre explicite	5	5
structură†	agregare pe mai multe niveluri, auto-îmbinare, anti-îmbinare, recursivitate, operații pe mulțimi, goluri și insule (<i>gaps and islands</i>)	5	7
măsură†	rapoarte, percentile, pivotare	4	3
temporal†	aritmetică de calendar, cohorte	4	4
ambigue	fără răspuns unic; punctate separat	7	7

Categoria *ambiguă* necesită o precizare metodologică distinctă, fiind singura evaluată după un criteriu calitativ. Aceste întrebări nu admit un răspuns SQL unic. De exemplu, „Care este venitul mediu per client?” permite trei interpretări valide în schema car_rental. Sistemul este evaluat nu pe

baza interogării emise, ci pe capacitatea de a *solicita clarificări* în loc de a selecta arbitrar o interpretare implicită. Patru dintre cele paisprezece întrebări sunt formulate în limba română, pentru a verifica dacă comportamentul se transferă între limbi.

3.6 Alegerea tehnologiilor

- **PostgreSQL** [17] **prin pgserver** [24]: executorul pornește o instanță PostgreSQL încorporată, fără Docker și fără instalare de sistem. Este condiția practică pentru ca rezultatele să fie reproductibile de un evaluator extern.
- **SQLite** [18] pentru BIRD — interogările de referință ale BIRD sunt scrise pentru SQLite. Portarea lor la PostgreSQL ar fi însemnat înlocuirea SQL-ului setului de evaluare cu SQL propriu, adică modificarea obiectului măsurat. A fost adăugat în schimb un executor SQLite.
- **llama.cpp** [23] pentru configurația locală, expus prin interfața compatibilă OpenAI, ceea ce permite configurațiilor A și C să folosească exact același cod client.
- **Agenți de linie de comandă ca subproces**: configurația B rulează agenții *așa cum sunt livrați*, cu promptul lor de sistem implicit. Ceea ce se măsoară este comportamentul real al produsului.

4. Testarea soluției și rezultate experimentale

Acest capitol prezintă protocolul de măsurare și rezultatele obținute. Structura urmează logica experimentului: mai întâi condițiile în care s-a măsurat (secțiunea 4.1), apoi rezultatul de ansamblu (secțiunea 4.2), apoi fiecare configurație în parte (secțiunile 4.3–4.5), apoi analizele care explică factorii determinanți ai rezultatelor obținute (secțiunile 4.6–4.7), apoi validarea externă (secțiunea 4.8) și, în final, limitele măsurătorii (secțiunea 4.9).

4.1 Metodologia de testare

4.1.1 Punerea în funcțiune

Cadrul de evaluare nu necesită instalare de sistem. Executorul PostgreSQL pornește o instanță încorporată prin pgserver, iar baza este creată din scripturile de inițializare la prima rulare. Modelele locale sunt servite de llama.cpp prin interfața compatibilă OpenAI. Cheia de acces pentru modelele din cloud este citită din mediu. O rulare completă se lansează cu o singură comandă (anexa B).

4.1.2 Parametrii de testare

Parametrii ficși ai tuturor măsurătorilor sunt sintetizați în tabelul 4.1.

Tabela 4.1: Parametrii ficși ai tuturor măsurătorilor.

Parametru	Valoare
Metrică	acuratețe de execuție; comparare de multimulțimi de tupluri; numele coloanelor ignorate; ordinea contează doar dacă întrebarea o cere
Temperatură	0 pentru toate modelele
Repetări	o rulare per celulă (vezi limitele din secțiunea 4.9)
Timp maxim per instrucțiune	15 s, impus în executor
Context / buget de ieșire, configurație C	16 384 / 8 192 tokeni
Hardware, configurație C	RTX 5070, 12 GiB VRAM; 31 GiB RAM; i9-14900K
Motor de inferență, configurație C	llama.cpp, compilare CUDA cb295bf
Preț de referință	tarife verificate la 23.08.2026, înregistrate în fiecare fișier de rezultate

4.1.3 Criterii de evaluare și punctare

Din cele 51 de întrebări car_rental, 44 sunt punctate pe acuratețe și 7 sunt ambigue, punctate separat (vezi secțiunea 4.7). Aceeași separare se aplică la adventureworks (49 + 7). Toate procente de acuratețe din acest capitol se raportează la populația punctată, iar numărările de erori din tabelele configurației C se raportează la aceeași populație: o precizare necesară, deoarece raportarea acelorași erori la întregul set de 51 de întrebări ar genera valori procentuale diferite.

O rulare a fost exclusă integral. claude-agent pe claude-fable-5 a atins limita de utilizare a contului la întrebarea 41; pentru ultimele 11 întrebări procesul a ieșit imediat cu eroare. Scorul brut de 75,0 % al acestui fișier clasifică 8 eșecuri de infrastructură drept răspunsuri eronate, motiv pentru care rularea **a fost exclusă din analiza comparativă**. Nicio altă rulare nu a avut vreun eșec de infrastructură.

4.2 Rezultatul de ansamblu

Figura 4.1 prezintă toate cele 18 rulări: cinci modele din cloud cu un singur apel, șapte configurații agentice și șase modele rulate local, măsurate pe car_rental, colorate după configurație.

Din analiza datelor reies trei observații principale:

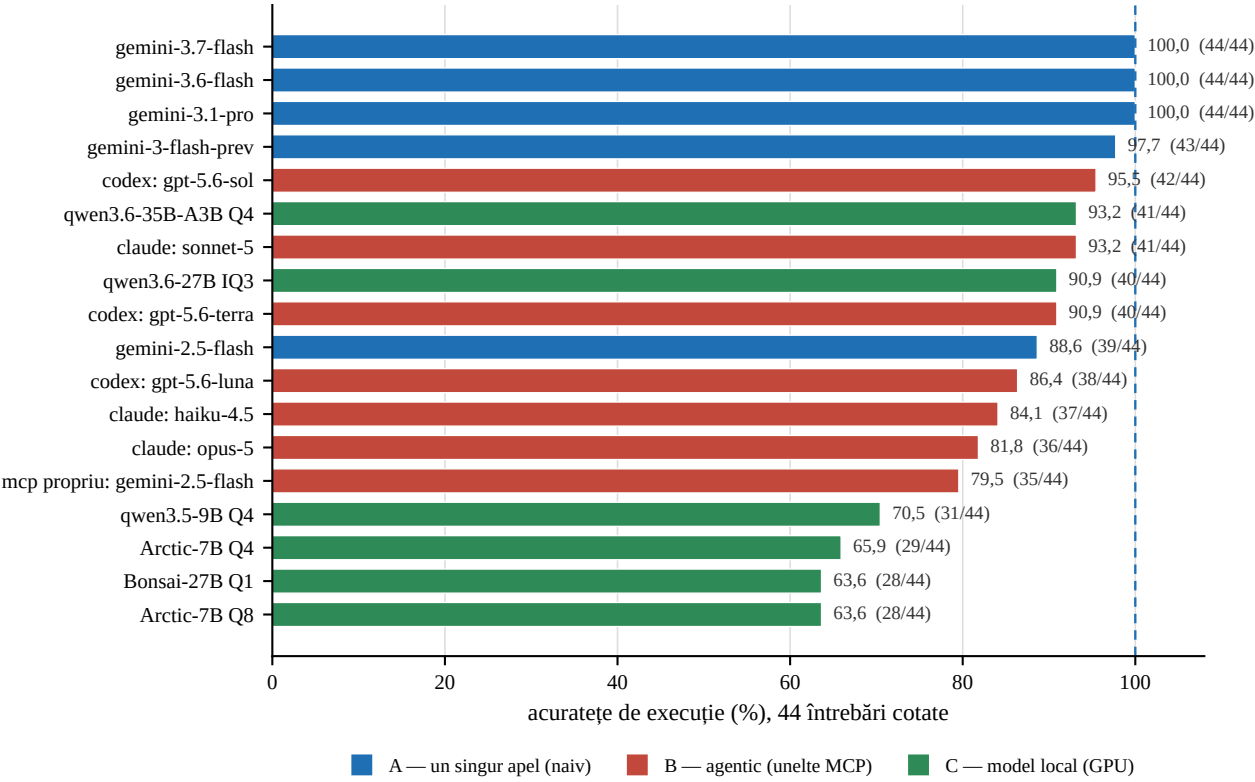


Figura 4.1: Acuratețea de execuție a tuturor sistemelor măsurate pe car_rental, 44 de întrebări punctate. Linia întreruptă marchează plafonul atins de configurația A. Niciun sistem agentic nu îl atinge.

1. **Primele patru sisteme din clasament folosesc toate configurația A**, cea mai simplă dintre cele trei. Cel mai bun sistem agentic răspunde corect la 42 din cele 44 de întrebări, reprezentând o performanță notabilă; totuși, trei configurații cu un singur apel ating scorul maxim de 44/44.
2. **Un model local de 35 de miliarde de parametri, rulat pe o placă de 12 GiB, obține 93,2 %**, exact scorul celui mai bun agent din familia Claude, fără ca datele să părăsească infrastructura locală.
3. **Ordinea nu urmează capacitatea modelelor.** În configurația agentică, modelul cel mai puternic al familiei Claude (Opus 5, 81,8 %) se clasează sub cel mediu (Sonnet 5, 93,2 %) și sub cel mic (Haiku 4.5, 84,1 %). Acest tipar reapare independent (vezi secțiunea 4.8).

4.3 Configurația A — un singur apel

Tabela 4.2: Configurația A: schema integrală, un apel. Costul este extrapolat din consumul de tokeni măsurat, la tarifele verificate la 23.08.2026.

Model	Acuratețe		Măsurat pe car_rental			
	car_rental	AdventureWorks	Latență med.	Tok. prompt	Tok. ieșire	\$/100 î.
gemini-3.7-flash	100,0 %	100,0 %	2,7 s	1 511	685	\$0,37
gemini-3.6-flash	100,0 %	98,0 %	6,4 s	1 511	1 079	\$0,52
gemini-3.1-pro	100,0 %	100,0 %	6,7 s	1 511	1 470	\$2,07
gemini-3-flash-prev	97,7 %	98,0 %	4,4 s	1 511	1 547	\$0,54
gemini-2.5-flash	88,6 %	91,8 %	1,9 s	1 511	654	\$0,21

Rezultatele obținute de configurația A sunt sintetizate în tabelul 4.2. Setul curent de întrebări separă decisiv gemini-2.5-flash și diferențiază ușor cele două modele intermediare, dar trei din cinci modele

obțin în continuare scorul maxim. Diferența dintre generații este reală și mare: de la gemini-2.5-flash la gemini-3.7-flash sunt +11,4 puncte procentuale pe car_rental, la un preț comparabil.

Un rezultat esențial pentru interpretarea capitolului este că extinderea schemei de la 5 951 la 37 019 caractere (de peste șase ori) *nu* degradează performanța configurației A. Cele două modele de vârf își mențin acuratețea de 100 % pe ambele baze de date. Ipoteza conform căreia transmiterea schemei integrale nu este scalabilă nu se verifică empiric la acest ordin de mărime al schemei.

4.4 Configurația B — agentic

Sunt măsurate două tipuri de sisteme. Primele șase rânduri din tabelul 4.3 sunt agenți de programare comerciali, rulați ca atare peste serverul MCP al proiectului. Ultimul rând este bucla proprie: un client minimal scris pentru această lucrare, care trimite întrebarea modelului gemini-2.5-flash, execută uneltele pe care acesta le cere și îi returnează rezultatele, până când modelul produce interogarea finală. Nu are prompt de sistem elaborat și nici strategie de explorare proprie.

Acest rând permite singura comparație complet controlată din capitol. gemini-2.5-flash este măsurat și în configurația A, pe aceleași 44 de întrebări și cu aceeași punctare, unde obține 88,6 %. Prin unelte, același model obține 79,5 %. **Diferența de 9,1 puncte este atribuibilă exclusiv arhitecturii**, fiindcă modelul, întrebările și baza de date sunt identice. Restul rândurilor din tabel amestecă efectul arhitecturii cu cel al modelului și al agentului comercial care îl conduce.

Tabela 4.3: Configurația agentică, car_rental. Costurile Claude sunt cifrele proprii ale *harness*-ului, care țin cont de memorarea contextului; la tarif de listă, Sonnet ar costa \$33,35 și Opus \$66,49 la 100 de întrebări.

Sistem	Corecte	Acuratețe	Iterații med.	Latență med.	Tok. prompt	\$/100 î.
codex: gpt-5.6-sol	42/44	95,5 %	2,3	23,6 s	67 848	nepublicat
claude: sonnet-5	41/44	93,2 %	4,4	12,4 s	162 170	\$9,44
codex: gpt-5.6-terra	40/44	90,9 %	2,1	18,0 s	64 916	nepublicat
codex: gpt-5.6-luna	38/44	86,4 %	2,5	22,0 s	70 978	nepublicat
claude: haiku-4.5	37/44	84,1 %	4,2	18,2 s	100 572	\$3,33
claude: opus-5	36/44	81,8 %	5,0	18,0 s	126 673	\$23,19
mcp propriu: gemini-2.5-flash	35/44	79,5 %	2,5	4,8 s	nemăsurat	nemăsurat
<i>referință:</i> configurația A, cea mai bună	44/44	100,0 %	0	2,7 s	1 511	\$0,37

4.4.1 Eficiența accesului prin unelte în raport cu linia de referință

Toate configurațiile agentice măsurate răspund corect la majoritatea întrebărilor (între 79,5 % și 95,5 %), deci accesul la baza de date prin unelte este o abordare care funcționează. Niciuna nu atinge însă reperul obținut cu un singur apel, iar diferența nu este marginală: 4,5 puncte pentru cel mai bun agent și 20,5 puncte pentru cel mai slab. Rezultatul se reproduce peste **două harness-uri complet diferite** (claude -p [15] și codex exec [16]), peste **șase modele** de la doi furnizori, plus configurația MCP proprie a proiectului, construită peste un al treilea furnizor. Nu este, prin urmare, o particularitate a unei implementări.

Explicația plauzibilă este aceea că arhitectura B nu măsoară capacitatea modelului, ci interacțiunea dintre comportamentul implicit al unui *harness* de programare și o sarcină de explorare a schemei. Un agent proiectat să exploreze temeinic un depozit de cod explorează temeinic și o bază de date, consumă din bugetul de atenție pe rezultate intermediare și ajunge la formularea interogării cu un context deja încărcat. Observația care susține această interpretare este că *modelul cel mai capabil consumă cele mai multe iterații* — Opus 5, cu 5,0 iterații în medie, este ultimul dintre modelele comerciale din această configurație.

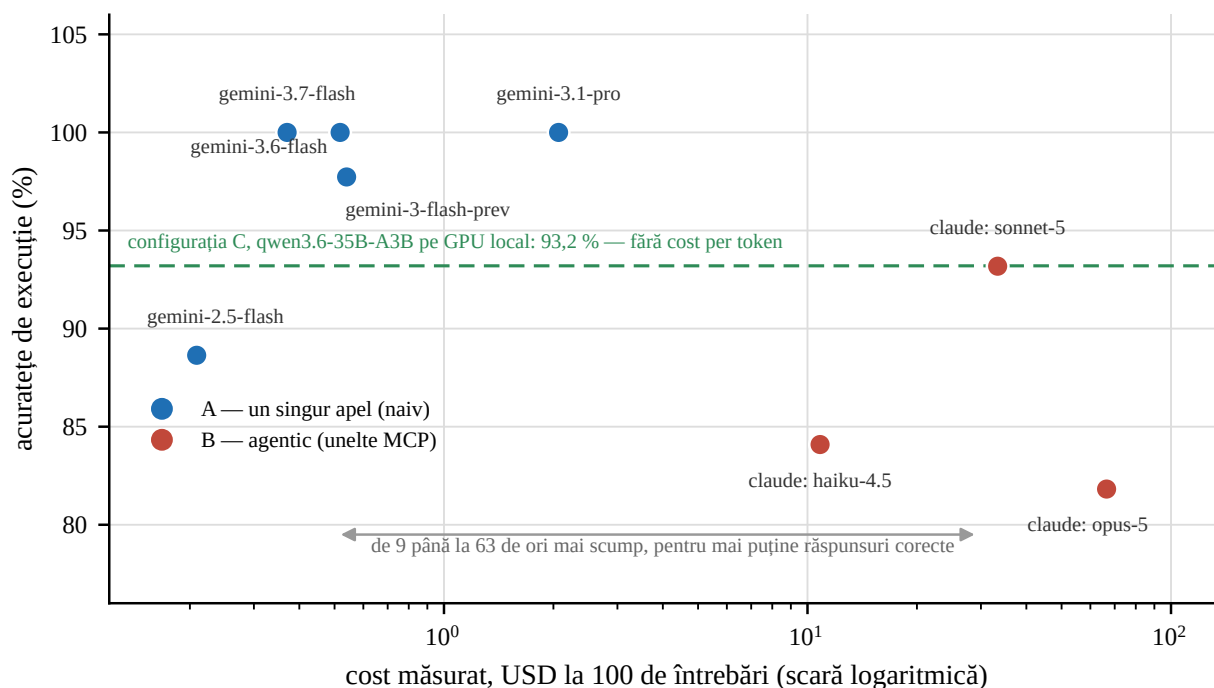


Figura 4.2: Costul măsurat în raport cu acuratețea de execuție. Configurațiile se diferențiază pe axa costului cu peste un ordin de mărime, în defavoarea abordării agentice.

4.4.2 Costul

Așa cum ilustrează figura 4.2, costul separă configurațiile mult mai puternic decât acuratețea: \$0,37 la 100 de întrebări pentru un singur apel, față de \$3,33–\$23,19 pentru variantele agentice: **un factor de cost de 9 până la 63 de ori mai mare, asociat unei rate inferioare de răspunsuri corecte**. Cauza este coloana de tokeni de prompt: 65 000 până la 162 000 în medie, față de 1 511, fiindcă un agent retrimite întregul context la fiecare iterație, peste promptul de sistem al *harness*-ului.

Rezultatele experimentale nu implică o deficiență intrinsecă a modelelor agentice; interpretarea susținută de date este că un agent generalist de programare nu reprezintă instrumentul optim pentru o sarcină deterministă, rezolvabilă mai eficient printr-un apel unic bine structurat. Distincția nu poate fi însă tranșată de acest experiment, care compară sisteme întregi, nu componente izolate.

4.5 Configurația C — modele rulate local

4.5.1 Acuratețea

Tabela 4.4: Configurația C: modele rulate local. Coloanele de eșecuri se raportează la cele 44 de întrebări punctate.

Model	Acuratețe		Eșecuri pe car_rental		
	car_rental	AdventureWorks	SQL inv.	fără SQL	trunchiat
qwen3.6-35B-A3B Q4	93,2 % (41/44)	85,7 % (42/49)	2	0	0
qwen3.6-27B IQ3	90,9 % (40/44)	nerulat	3	0	0
qwen3.5-9B Q4	70,5 % (31/44)	67,3 % (33/49)	0	5	5
Arctic-7B Q4	65,9 % (29/44)	55,1 % (27/49)	10	0	0
Bonsai-27B Q1	63,6 % (28/44)	69,4 % (34/49)	6	6	6
Arctic-7B Q8	63,6 % (28/44)	55,1 % (27/49)	9	0	0

Rezultatele privind acuratețea sunt prezentate în tabelul 4.4. Rezultatul principal al acestei configurații este primul rând: un model MoE de 35 de miliarde de parametri, cuantizat pe 4 biți, atinge 93,2 % pe `car_rental` — egal cu cel mai bun agent Claude — și 85,7 % pe o schemă de 68 de tabele. Pentru un sistem care prelucrează date de întreprindere, echivalența funcțională cu un serviciu din cloud, obținută fără ca datele să părăsească infrastructura locală, reprezintă o concluzie cu consecințe practice directe.

4.5.2 Constrângerea de performanță: modele dense versus MoE

Tabela 4.5: Configurația C: încadrarea în memoria plăcii și consecința asupra debitului. VRAM este ocuparea măsurată la încărcare.

Model	GGUF (GiB)	VRAM (MiB)	Cum încape	tok/s	Latență med.	Durată, 44 î.
Arctic-7B Q4	4,4	6 055	rezident integral	113	4,0 s	4 min
qwen3.5-9B Q4	5,2	6 702	rezident integral	91	8,7 s	15 min
Arctic-7B Q8	7,5	9 067	rezident integral	72	6,1 s	5 min
Bonsai-27B Q1	3,5	6 284	rezident integral	53	47,2 s	50 min
qwen3.6-35B-A3B Q4	20,6	9 620	experți în RAM	50	39,5 s	37 min
qwen3.6-27B IQ3	11,2	10 438	straturi parțiale	8	268,5 s	247 min

În tabelul 4.5 sunt sintetizate ocuparea memoriei și debitele de generare obținute. Ultimele două rânduri evidențiază o discrepanță arhitecturală remarcabilă. Modelul cu *cel mai mare volum ocupat pe disc* (20,6 GiB, depășind capacitatea de 12 GiB a memoriei VRAM) atinge un debit de generare de 50 tokeni/secundă, în timp ce un model dens de dimensiune aproximativ la jumătate (11,2 GiB) obține doar 8 tokeni/secundă. Modelul MoE este **de peste șase ori mai rapid** și, totodată, mai exact (93,2 % față de 90,9 %).

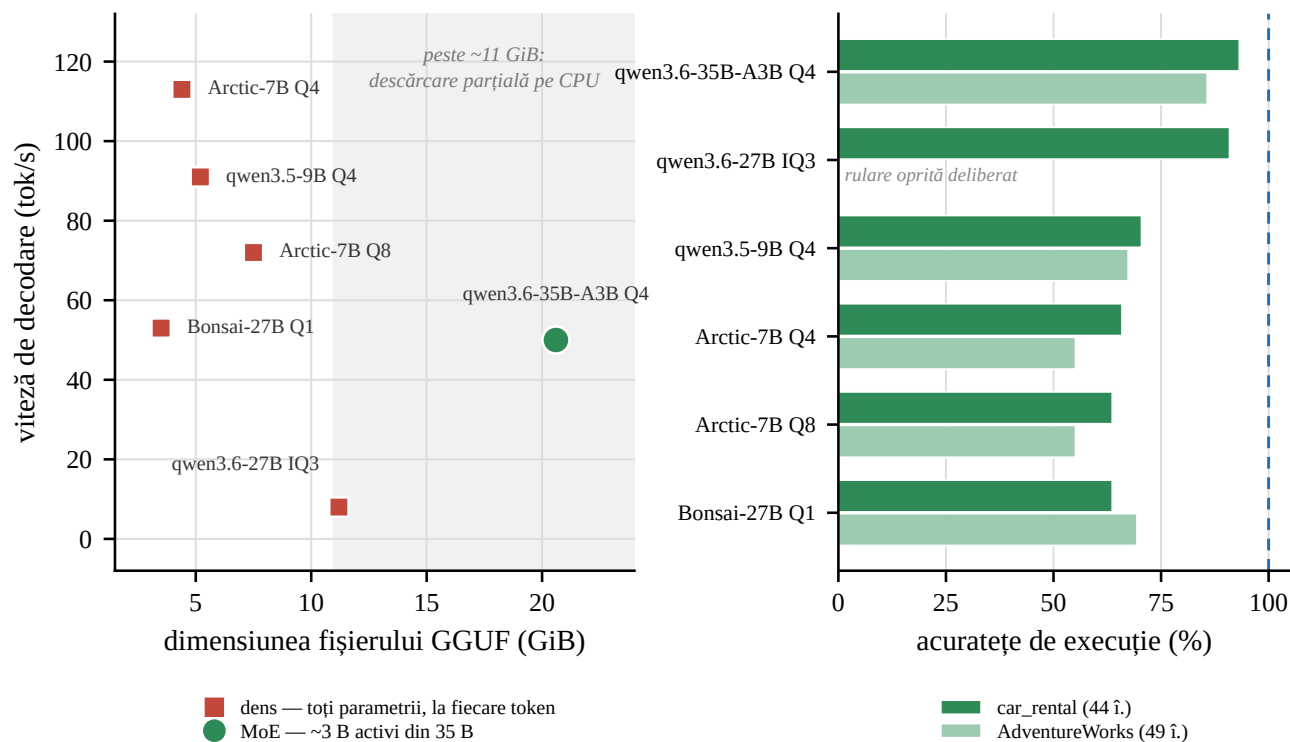


Figura 4.3: Stânga: debitul de decodare nu este o funcție descrescătoare de dimensiunea fișierului. Punctul verde este singurul model MoE. Dreapta: acuratețea aceluiași modele pe cele două baze PostgreSQL.

Comportamentul este ilustrat în figura 4.3. Explicația este arhitecturală. Un model dens citește toți parametrii pentru fiecare token generat; când nu încapă în VRAM, aproximativ 5 GiB traversează

magistrala PCIe *la fiecare token*. Un model MoE citește doar aproximativ 3 din cei 35 de miliarde de parametri, ceea ce permite păstrarea experților inactivi în memoria RAM a sistemului cu o penalizare redusă de latență. Pe sisteme cu resurse hardware limitate, **criteriul de selecție nu este dimensiunea statică a modelului, ci volumul de parametri transferat per token generat**.

Consecința practică este că modelul dens de 27B nu este o configurație utilizabilă pentru un sistem interactiv, *la orice acuratețe*: 247 de minute pentru 44 de întrebări înseamnă aproximativ 5,6 minute per întrebare.

4.5.3 Rezultate experimentale secundare

Cuantizarea nu explică diferențele de acuratețe. Arctic-Text2SQL-R1 a fost măsurat de două ori, cu aceleași greutate: varianta pe 8 biți (7,5 GiB) obține un scor *mai mic* decât cea pe 4 biți (4,4 GiB) pe car_rental (63,6 % față de 65,9 %) și identic pe AdventureWorks, rulând totodată la 64 % din viteză. Dublarea numărului de biți nu a adus niciun beneficiu.

Problema modelului specializat este dialectul, nu raționamentul. Din cele 10 erori de execuție ale modelului Arctic pe car_rental, **8 sunt funcții SQLite pe care PostgreSQL nu le are** — strftime (de cinci ori), date(...) folosit ca constructor (de două ori) și julianday. Numărul este o limită inferioară: clasificarea se face după numele funcției, așa că o a noua eroare, operator does not exist: date - bigint (aritmetică de tip dată minus întreg, tot o particularitate SQLite) nu este numărată. Modelul a fost antrenat pe Spider și BIRD, ambele livrate în SQLite, și emite funcții de dată SQLite în ciuda unui prompt care specifică explicit „PostgreSQL” și a unei scheme redată în tipuri PostgreSQL. Dacă fiecare astfel de eșec ar fi rescris mecanic în echivalentul PostgreSQL, modelul ar ajunge la 84,1 %. Această valoare arată cât din decalaj se explică prin dialect și *nu este un scor obținut de model*: acuratețea lui măsurată rămâne 65,9 %.

Aceasta este o observație cu valoare generală: **o poziție în clasament nu se transferă între motoare de baze de date**. Eticheta „model specializat” descrie un dialect la fel de mult ca o sarcină: aspect ce nu este evidențiat în clasamentele majore, deoarece acestea evaluează predominant pe dialectul SQLite.

4.6 Analiza factorilor de dificultate

Tabela 4.6: Acuratețea agregată pe grupuri de întrebări, car_rental, mediată peste toate rulările fiecărei familii de configurații (%).

Familie	simple	agreg.	join	subint.	fereastră	structură	măsură	temporal
A — un singur apel	100,0	100,0	100,0	100,0	96,0	92,0	95,0	90,0
B — agentic	95,2	87,3	85,7	78,6	85,7	85,7	92,9	89,3
C — local	97,2	94,4	86,7	80,6	60,0	43,3	70,8	33,3

Tabelul 4.6 și figura 4.4 ilustrează aceleași concluzii din două perspective complementare. Grupurile *simple* și *agregare* (exact cele pe care le conțin seturile de evaluare clasice) nu mai separă niciun sistem din configurațiile A și B. Puterea de discriminare s-a mutat în cea mai mare parte în grupurile adăugate în etapa a doua: configurația locală scade de la 97,2 % pe *simple* la 43,3 % pe *structură* și 33,3 % pe *temporal*. Excepția o constituie configurația agentică, care pierde cel mai mult pe *subinterogare* (78,6 %), o categorie din prima etapă: ceea ce confirmă observația din secțiunea 4.4, că această configurație nu eșuează pe aceleași dimensiuni ca celelalte două.

Aceasta este și dovada că întrebările adăugate testează ceea ce au fost proiectate să testeze: fiecare punct pe care îl pierde gemini-2.5-flash se află într-unul dintre grupurile noi, iar cele patru categorii inițiale rămân răspunse perfect.

Inspecția manuală a interogărilor greșite, peste toate rulările, identifică trei moduri de eșec recurente, detaliate în tabelul 4.7:

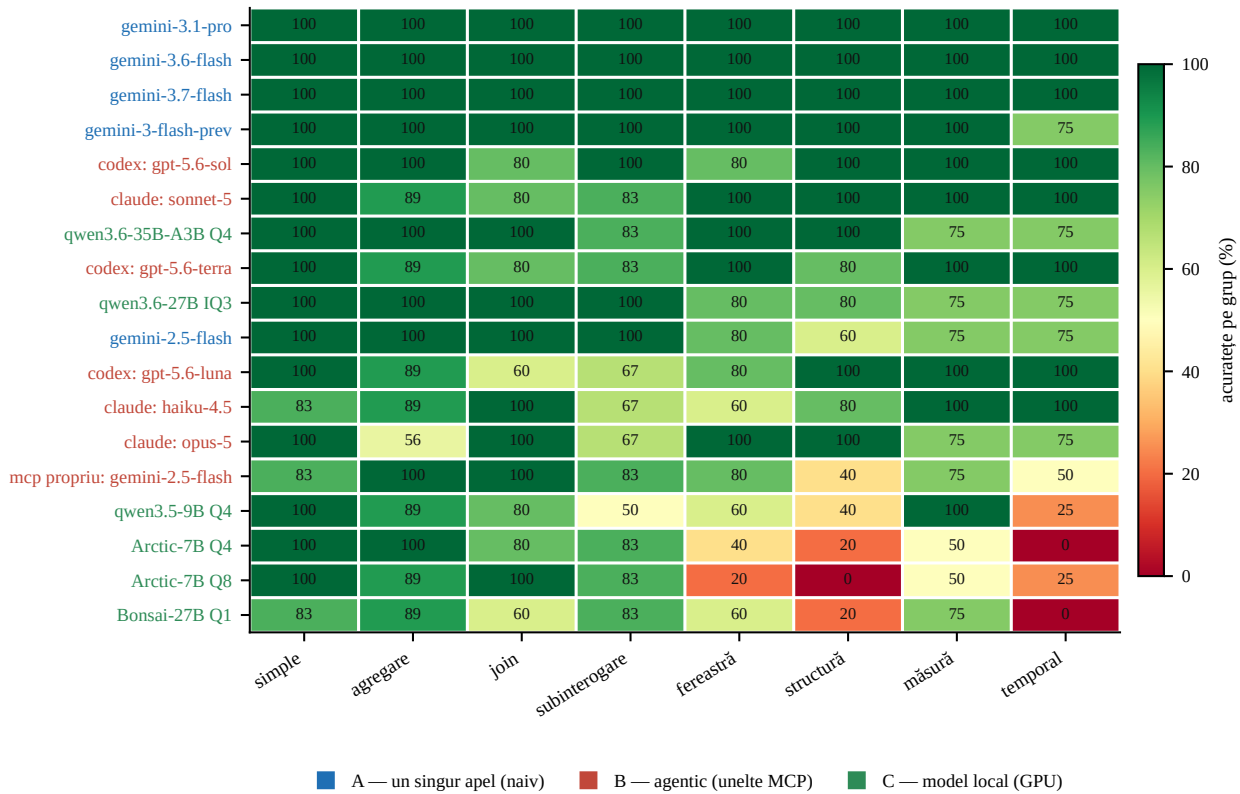


Figura 4.4: Acuratețea pe grupuri de întrebări, per sistem. Culoarea etichetei indică configurația. Categoriile din stânga prezintă un grad ridicat de saturație, în timp ce categoriile din dreapta evidențiază o capacitate puternică de discriminare.

Tabela 4.7: Modurile de eșec recurente, identificate prin reexecutarea manuală a interogărilor greșite.

Mod de eșec	Descrierea manifestării
Semantica cadrului de fereastră	Filtrarea pe anul de raportare este aplicată <i>înainte</i> de funcția fereastră, ceea ce trunchiază cadrul peste care se calculează
Utilizarea greșită a dialectului	round(double precision, integer) does not exist — în PostgreSQL, varianta cu două argumente cere numeric. Este exact a doua subclasă de erori ca frecvență din Spider 2.0 (10,3 %) [3]
Ancorarea cohorței	Cohorta este ancorată la data evenimentului greșit: cazul de eșec emblematic din Spider 2.0

4.7 Ambiguitatea, singura dimensiune nesaturată

Cele șapte întrebări ambigue nu au un răspuns corect. Sistemul este punctat pe faptul dacă *a cerut* o clarificare, rezultatele fiind sintetizate în tabelul 4.8 și figura 4.5.

Tabela 4.8: Câte rulări din fiecare familie au cerut clarificare, per întrebare, car_rental. Cinci rulări cu un singur apel, șapte agentice, șase locale.

Î.	Textul întrebării	Config. A	Config. B	Config. C
q30	„Hello, what can you do?”	5/5	4/7	6/6
q28	„Show me the good vehicles.”	4/5	6/7	2/6
q29	„How is the business doing?”	3/5	4/7	1/6
q27	„Care sunt cei mai buni clienți?”	1/5	4/7	1/6
q49	„Care sucursală a adus cei mai mulți bani în 2024?”	0/5	3/7	1/6
q51	„How many cars were rented last month?”	0/5	3/7	0/6
q50	„What is our average revenue per client?”	0/5	2/7	0/6

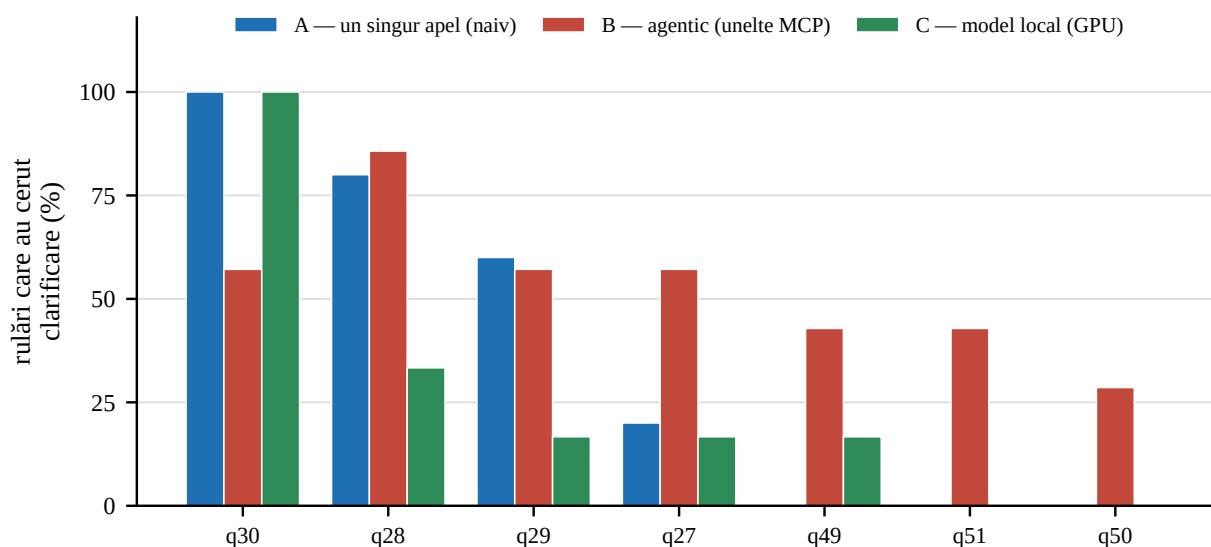


Figura 4.5: Rata de solicitare a clarificării, pe întrebare și pe familie de configurații. Ordinea pe axa orizontală este descrescătoare după rata medie.

Din aceste măsurători se desprind trei concluzii metodologice esențiale:

1. Ambiguitatea care se anunță este detectată; cea care se ascunde nu este. O întrebare care nu conține nicio metrică („Cum merge afacerea?”) declanșează o cerere de clarificare. O întrebare care *pare* răspunzabilă nu o declanșează. „Care este venitul mediu per client?” admite trei numitori apărăbili în această schemă, iar **toate cele cinci modele cu apel unic au selectat implicit una dintre interpretări, fără a semnaliza ambiguitatea.**

2. Este o limitare intrinsecă de capacitate, nu o consecință a complexității sintactice. Modelele nu eșuează în a scrie SQL-ul; eșuează în a observa că întrebarea nu determină ce SQL trebuie scris. Spre deosebire de tabelul de acuratețe, această dimensiune *nu* este saturată: este singura din întreaga lucrare care nu este.

3. Accesul prin unelte inversează rezultatul. În regim agentic, mai multe modele manifestă o disponibilitate semnificativ mai mare de a solicita clarificări: configurația B este singura care obține rezultate nenule la q50 și q51, iar la q49 obține cel mai bun rezultat dintre cele trei familii: exact întrebările la care configurația A obține 0/5. Posibilitatea de a *vedea* cei trei numitori candidați crește probabilitatea ca modelul să identifice subspecificarea întrebării și să ceară lămuriri. **Aceasta este singura dimensiune pe care configurația agentică o depășește pe cea de bază**, și este exact dimensiunea pe care se sprijină mecanismul de clarificare din sistemul propriu.

Rezervă metodologică: punctarea este binară. Un model care răspunde *enunțând explicit ipoteza asumată* este punctat identic cu unul care răspunde tăcut. Înainte ca această cifră să devină un rezultat principal, distincția ar trebui să devină o a treia stare.

4.8 Validarea externă pe BIRD-SQL

Experimentele descrise anterior au evaluat performanța sistemelor pe baza întrebărilor formulate în term de autorul lucrării. Baza de date AdventureWorks pune la dispoziție o schemă relațională complexă, dar nu include un set standardizat de întrebări; din investigațiile întreprinse, aceasta nu este utilizată în seturile de referință consacrate, astfel că întrebările din secțiunile 4.2–4.7 au fost elaborate specific pentru prezenta lucrare. Aceasta este o limită de validitate pe care lucrarea o declară explicit, iar secțiunea de față este răspunsul la ea.

BIRD-SQL [2] este setul de evaluare de referință pentru text-to-SQL ancorat în date: 1 534 de întrebări peste 11 baze SQLite, cu interogări de referință publicate, metrică publicată și clasament public. Configurația este *aceeași* configurație A din secțiunea 4.3, schema integrală, un apel, fără recuperare și fără reparare. Se schimbă doar baza de date și dialectul.

4.8.1 Rezultate

Rezultatele comparative pe setul BIRD-SQL dev sunt prezentate în tabelul 4.9.

Tabela 4.9: BIRD-SQL dev, toate cele 1 534 de întrebări, fără eșantionare. Coloanele de dificultate folosesc etichetele proprii ale BIRD, pe care cadrul de evaluare nu le vede niciodată.

Rulare	Acurat.	Corecte	simple (860)	moderate (443)	dificile (231)	inv.	f.SQL	Lat.	Cost
gemini-3.7-flash + dicționar	64,9 %	996/1534	74,1 %	63,2 %	34,2 %	10	7	3,1 s	\$7,31
gemini-3.7-flash	63,6 %	976/1534	73,3 %	61,6 %	31,6 %	12	8	3,6 s	\$5,64
gemini-3.1-pro-preview	62,1 %	952/1534	71,3 %	59,4 %	32,9 %	1	21	10,5 s	\$29,10
gemini-2.5-flash	55,7 %	855/1534	64,9 %	52,6 %	27,7 %	8	12	1,4 s	\$2,76

Cheltuiala totală a fost de \$44,81. Toate cele patru rulări s-au încheiat complet, fără niciun eșec de infrastructură.

4.8.2 Eliminarea saturației de evaluare prin setul BIRD

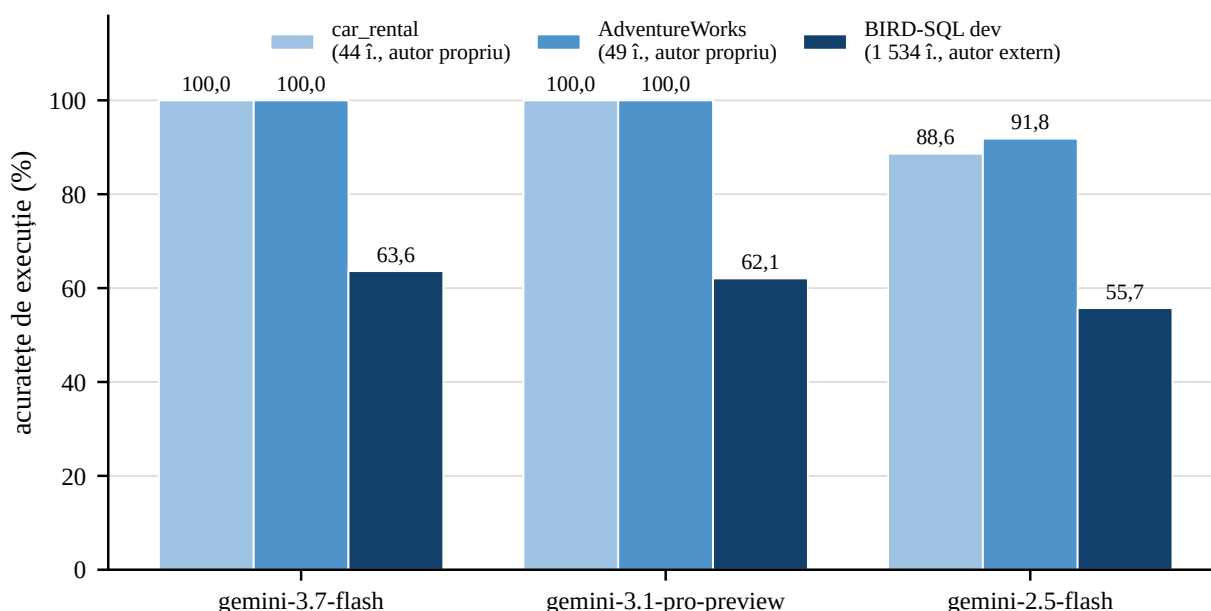


Figura 4.6: Aceleași trei modele, sub aceeași configurație, pe cele trei seturi de întrebări. Plafonul de performanță (saturația) este depășit prin evaluarea pe baza întrebărilor concepute independent.

Acesta este rezultatul central al secțiunii, ilustrat în figura 4.6, iar scăderea scorurilor absolute reflectă un beneficiu metodologic, permițând discriminarea obiectivă a sistemelor. Trei modele care pe întrebările proprii obțineau 100 %, deci erau indistinctibile între ele, coboară pe BIRD într-o bandă de 56–65 %, suficient de largă pentru a le ordona. **Capacitatea de discriminare a sistemelor a fost restabilită prin utilizarea unui set de întrebări independent**, și nu prin sporirea complexității sintactice, abordare care s-a dovedit inefficientă în secțiunea 4.3.

Gradientul de dificultate este monoton în fiecare dintre cele patru rulări, peste etichetele proprii ale BIRD, pe care cadrul de evaluare nu le primește niciodată. Patru rulări independente care reproduc aceeași ordonare constituie totodată dovada că modulul de punctare este corect. Distribuția performanței pe niveluri de dificultate și pe cele 11 baze de date este ilustrată în figura 4.7.

4.8.3 Comparația de performanță: modelul pro versus modelul flash

gemini-3.1-pro-preview obține o acuratețe *inferioară* modelului gemini-3.7-flash, la un cost de 5 ori mai ridicat și o latență de 3 ori mai mare: deși cele două fuseseră indistinctibile pe ambele baze proprii.

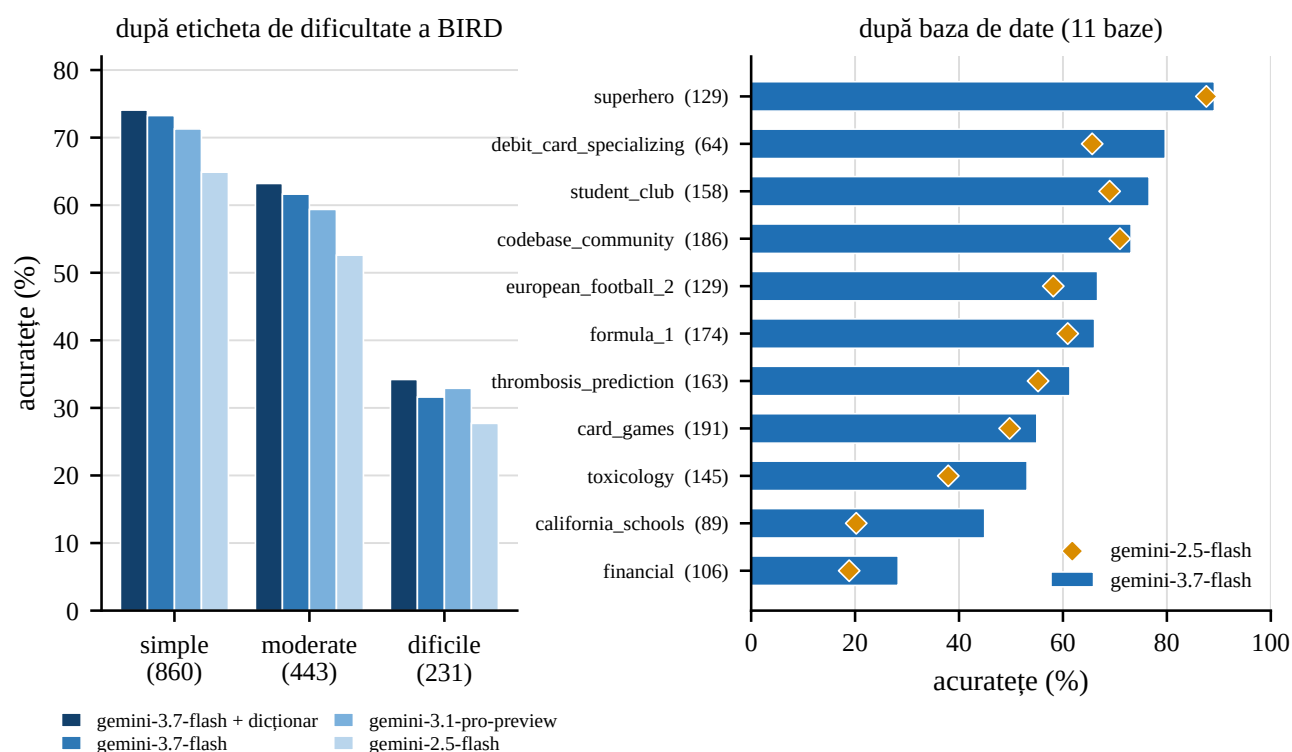


Figura 4.7: Stânga: acuratețea pe cele trei niveluri de dificultate ale BIRD. Dreapta: dispersia pe cele 11 baze de date, cu numărul de întrebări în paranteză.

Descompunerea pe dificultate explică de ce: modelul pro câștigă +1,3 puncte pe întrebările *dificile*, dar pierde 2,0 și 2,2 puncte pe cele *simple* și *moderate*, care reprezintă 85 % din set.

Profilul erorilor confirmă această concluzie: modelul pro produce 1 interogare invalidă și 21 de refuzuri, în timp ce flash produce 12 invalide și 8 refuzuri. Modelul pro manifestă o prudență sporită (generând refuzuri de răspuns), penalizată mai sever în acuratețea de execuție decât generarea de sintaxă invalidă de către modelul flash.

Este exact tiparul din secțiunea 4.4, reprodus într-un cadru complet diferit: acolo, Opus 5 (81,8 %) se clasa sub Sonnet 5 (93,2 %) și sub Haiku 4.5 (84,1 %). **Capacitatea modelului nu ordonează rezultatele, în niciunul dintre cele două experimente.**

4.8.4 Impactul dicționarului de date asupra acurateței

BIRD livrează un dicționar de date per coloană pe care îl ține deliberat în afara DDL-ului. În baza financial, o coloană a cărei linie CREATE TABLE conține clauza A3 TEXT not null are intrarea de dicționar „region”, de neghicit din schemă.

Rulările din tabelul 4.9 folosesc *doar DDL brut*, ceea ce păstrează contractul de prompt identic cu cel din secțiunea 4.3. Rularea suplimentară adaugă dicționarul de date păstrând neschimbați toți ceilalți parametri, rezultatele fiind sintetizate în tabelul 4.10:

Tabela 4.10: Efectul dicționarului de date, gemini-3.7-flash.

	Tokeni de prompt	Acuratețe
DDL brut	1 062	63,6 %
DDL + dicționar	2 600	64,9 %

Un spor de +1,3 puncte procentuale obținut cu un prompt de 2,4 ori mai mare. Efectul nu reprezintă un câștig pur de precizie: la nivelul fiecărei întrebări, dicționarul *a corectat 64 de cazuri și a provocat erori în alte 44*, soldul net fiind de +20. Pe baze de date individuale, bilanțul net ajunge

de la +10 (toxicology) și +5 (student_club), acolo unde numele coloanelor sunt opace, la -3 (financial, superhero), acolo unde cei 1 538 de tokeni suplimentari de proză aglomerează schema cu informație plauzibilă, dar irelevantă. Pe card_games efectul se anulează exact: dicționarul corectează 9 întrebări și induce erori în alte 9.

Consecința pentru lucrare este că valoarea de 63,6 % se află la mai puțin de un punct de configurația comparabilă cu clasamentul, deci poate fi citată alături de cifrele publicate, cu o notă de subsol, iar promptul rămâne identic cu cel folosit pe celelalte două baze, ceea ce menține toate cele trei seturi în același experiment.

4.8.5 Dispersia între baze de date

Tabela 4.11: Acuratețea pe fiecare dintre cele 11 baze BIRD.

Baza de date	Î.	3.7-flash	3.1-pro	2.5-flash
superhero	129	89,1 %	86,0 %	87,6 %
debit_card_specializing	64	79,7 %	75,0 %	65,6 %
student_club	158	76,6 %	77,2 %	69,0 %
codebase_community	186	73,1 %	74,7 %	71,0 %
european_football_2	129	66,7 %	64,3 %	58,1 %
formula_1	174	66,1 %	64,9 %	60,9 %
thrombosis_prediction	163	61,3 %	59,5 %	55,2 %
card_games	191	55,0 %	54,5 %	49,7 %
toxicology	145	53,1 %	45,5 %	37,9 %
california_schools	89	44,9 %	43,8 %	20,2 %
financial	106	28,3 %	28,3 %	18,9 %

Așa cum reiese din tabelul 4.11, dispersia este de 61 de puncte între baze, pe un singur model: mult mai mare decât cele 8 puncte care separă modelele între ele. **Particularitățile bazei de date influențează performanța într-o măsură mai mare decât alegerea modelului.** Baza financial, la 28,3 %, prezintă cel mai ridicat grad de opacitate al identificatorilor de coloane (A2, A3, A11).

4.8.6 Validarea manuală a eșecurilor de execuție

Un scor de 63,6 % pentru un model care atinge 100 % pe întrebările proprii ar putea sugera prezența unor anomalii metodologice în cadrul de evaluare. În consecință, un eșantion stratificat de 30 de interogări eșuate a fost reexecutat și verificat manual, fiind clasificat în tabelul 4.12:

Tabela 4.12: Clasificarea manuală a eșecurilor dintr-un eșantion stratificat.

Cauză	Nr.
Răspuns greșit autentic (formă corectă, valori greșite)	8
Set de coloane greșit / număr de rânduri greșit	4
SQL-ul modelului nu s-a executat	1
Artefact de punctare (confirmat drept corect de metrica oficială BIRD)	1

Un singur caz din cele 14 eșecuri analizate reprezintă un artefact de punctare. Metrica oficială BIRD compară *mulțimi* de tupluri; cadrul de evaluare de față compară *multimulțimi*, deci multiplicitatea rândurilor duplicate contează pentru cadrul de evaluare de față, dar nu și pentru metrica oficială BIRD. Adoptarea regulii exacte a BIRD mută eșantionul de la 53,3 % la 56,7 %, o singură întrebare. Aceasta este singura divergență între cele două metrice și este declarată aici tocmai pentru că afectează comparabilitatea cu clasamentul.

Celelalte 13 sunt modurile de eșec canonice descrise de autorii BIRD — ancorare în valori și literalii sensibili la majuscule, nu construcție de SQL:

```
gold WHERE A3 = 'north Bohemia' -> 2260
pred WHERE A3 = 'North Bohemia' -> 0 <- 'n' mic in date

gold WHERE Location = 'New York' -> 15
pred WHERE Location LIKE '%New York%' -> 295

gold WHERE atom_id LIKE '%_19' -> 498
pred WHERE atom_id = '19' -> 0 <- id-urile sunt TR000_19
```

Listing 4.1: Trei eșecuri reprezentative, cu numărul de rânduri returnate de fiecare interogare.

4.8.7 Note de metodă

- **Toate cele 1 534 de întrebări sunt incluse**, fără eșantionare.
- **Validarea referințelor a rulat prima.** Toate cele 1 534 de interogări de referință au fost executate înainte de orice apel către un model. *Niciuna nu returnează zero rânduri*, deci mecanismul de refuz al referințelor triviale rămâne armat. Trei depășesc limita de 15 s și sunt înregistrate ca eroare de referință, nu ca eșec al modelului; reducând scorul fiecărei rulări cu 3 puncte.
- **SQLite, nu tradus.** Interogările de referință BIRD sunt scrise pentru SQLite. Portarea lor la PostgreSQL ar fi substituit SQL-ul propriu celui al setului de evaluare.
- **Câmpul de cunoaștere externă este inclus**, așa cum cere definiția sarcinii din BIRD; întrebările nu sunt rezolvabile fără el.
- **Comparare fără ordine**, conform metricii proprii a BIRD.

4.9 Amenințări la adresa validității și limitări

4.9.1 Anomalii identificate în cadrul de evaluare și remedierea acestora

Rularea modelelor locale a evidențiat trei anomalii latente. Toate trei *diminuau acuratețea exclusiv în cazul modelelor locale*: o formă de eroare sistematică ce risca să conducă la concluzii eronate, acestea fiind sintetizate în tabelul 4.13.

Tabela 4.13: Defectele descoperite, efectul lor și remedierea.

Defect	Efect	Remediere
Ipoază asupra formatului de ieșire. Extractorul de SQL elimina blocul marcat doar dacă răspunsul începea cu el	Arctic își narează derivarea și pune interogarea într-un bloc <i>final</i> , deci întreaga narațiune era executată ca predicție: 0/2 pe întrebări răspunse corect	Se ia <i>ultimul</i> bloc marcat. Pentru ambele forme produse de modelele din cloud rezultatul este identic la nivel de octet, rezultatele configurației A rămânând complet neschimbate
Limitarea lungimii de generare clasificată ca eșec al modelului. Limita de ieșire era de 4 096 de tokeni	Un model care raționează înainte de a răspunde consumă cea mai mare parte a bugetului ajungând la interogare: qwen3.5-9B era trunchiat la 6 din 44	Ridicare la 8 192. <i>Întregul set de rulări locale a fost reexecutat integral</i> , asigurând condiții experimentale riguros uniforme pentru toate rezultatele configurației C
Trunchierea răspunsului clasificată drept gestiune a ambiguității. Această configurație calcula „a cerut clarificare” ca „nu a produs SQL”	Șase răspunsuri trunchiate erau contorizate drept cereri de clarificare, denaturând metrica de evaluare a ambiguității	Detectarea explicită a motivului de oprire, înregistrare separată, excludere din coloana de clarificări

Fișierele generate anterior acestei remedieri nu includ câmpul respectiv, absența sa funcționând ca indicator de învechire ce determină excluderea automată a rulării din agregările curente.

4.9.2 Acoperire

Situația măsurătorilor și a limitelor de acoperire este centralizată în tabelul 4.14.

Tabela 4.14: Sinteza acoperirii experimentale și a limitărilor acesteia.

Celulă	Stare
Configurație A, 5 modele × 2 baze	măsurat
Configurație B, 7 sisteme, car_rental	măsurat
Configurație C, 6 modele, car_rental; 5 modele, adventureworks	măsurat
Configurație A, 4 rulări, BIRD-SQL dev (1 534 î.)	măsurat
Configurație B pe adventureworks	nerulat — nu se știe dacă penalizarea agentică crește cu dimensiunea schemei
Configurație C pe BIRD	nerulat — circa 19 ore doar pentru modelul rar
claude-agent pe claude-fable-5	parțial, exclus (vezi secțiunea 4.1)
Conducta RAG proprie (configurația din secțiunea 3.3.1)	nemăsurată față de setul curent de întrebări , detaliat în subsecțiunea 4.9.3
Studiu de varianță pe rulări repetate	învechit

4.9.3 Considerații privind evaluarea conduitei RAG proprii

Conducta RAG descrisă în secțiunea 3.3.1 **nu apare în niciun tabel din acest capitol**. Ultima ei măsurătoare a fost făcută față de un set de întrebări ulterior înlocuit, iar mecanismul de amprentare o marchează, corect, drept învechită. Alternativa ar fi fost citarea unei cifre obținute pe alte întrebări decât cele față de care sunt raportate toate celelalte sisteme, cu riscul de a sugera o comparabilitate inexistentă.

Această delimitare este asumată explicit în cadrul metodologic: **lucrarea nu își propune să demonstreze că soluția proprie depășește cele trei configurații măsurate**. Obiectivul validat experimental îl constituie analiza comparativă riguroasă dintre arhitecturile de generare, ale cărei concluzii sunt independente de conducta proprie. Reluarea acestei măsurători este prima poziție din direcțiile de dezvoltare (secțiunea 5.3).

4.9.4 Limite cunoscute ale măsurătorii

- **Autorul întrebărilor.** Două din cele trei seturi sunt scrise de autorul sistemului. Secțiunea 4.8 este contramăsura adoptată.
- **Scara schemelor.** Bazele au 9, 68 și, la BIRD, între 3 și 13 tabele. Concluziile acestei lucrări nu pot fi extrapolate direct la depozite masive de date (sute de tabele), regim studiat de Spider 2.0 [3] și BEAVER [4].
- **Metrica acceptă răspunsul corect dintr-un motiv greșit.** Compararea seturilor de rezultate nu poate diferenția o interogare semantic corectă de una care returnează accidental aceleași tupluri.
- **Ambiguitatea este punctată binar** (vezi rezerva din secțiunea 4.7).
- **O singură rulare per celulă.** Un studiu anterior de varianță a stabilit o dispersie de 3,8 puncte pe șase rulări, dar *pe setul vechi de întrebări*. Prin urmare, **diferențele sub aproximativ 4 puncte pe 44 de întrebări trebuie tratate ca zgomot** până la refacerea acelu studiu. Aceasta nu afectează rezultatele principale, care sunt despărțite de 20 până la 45 de puncte, dar afectează ordonarea fină din interiorul fiecărei configurații.

5. Concluzii

5.1 Contribuții personale

1. **Un cadru de evaluare reproductibil pentru sisteme text-to-SQL**, care punctează prin execuție reală, izolează o singură variabilă între rulări comparate, înregistrează costul alături de acuratețe și își detectează singur rulările învechite prin amprentarea setului de întrebări. Mecanismul de amprentare și-a dovedit utilitatea practic, prin interceptarea unei rulări învechite la generarea figurilor acestei lucrări.
2. **O măsurătoare a fezabilității accesului agentic la baza de date**, sub protocol controlat, peste două *harness*-uri independente și șase modele, raportată la două repere obținute în aceleași condiții.
3. **Două seturi de întrebări construite pe categorii derivate din taxonomia de erori publicată în literatură**, incluzând o categorie de întrebări deliberat ambigue, punctate pe cererea de clarificare. Spre deosebire de seturile dedicate ambiguității [12, 13], acestea măsoară comportamentul în aceeași rulare cu acuratețea.
4. **Un executor cu două motoare** (PostgreSQL și SQLite), care a permis validarea externă pe BIRD fără a traduce interogările de referință ale setului de evaluare, adică fără a modifica obiectul măsurat.
5. **Un sistem funcțional de interogare în limbaj natural**, cu conductă de recuperare, validare pe arbore sintactic, execuție read-only și generare de rapoarte.

5.2 Analiza rezultatelor

Accesul agentic la baza de date funcționează, dar nu la nivelul pe care îl sugerează argumentul din spatele lui. Toate configurațiile agentice măsurate produc interogări corecte pentru majoritatea întrebărilor (79,5–95,5 %), deci abordarea este fezabilă. Niciuna nu atinge însă reperul obținut cu un singur apel, iar costul este de 9 până la 63 de ori mai mare. Rezultatul se reproduce peste două *harness*-uri independente, șase modele și trei furnizori, deci nu este un accident de implementare. Predicția intuitivă (că un model care poate inspecta baza de date va scrie interogări mai bune) nu se verifică în condițiile măsurate. O delimitare metodologică importantă este că evaluarea vizează agentul în configurația sa nativă: datele nu indică o deficiență intrinsecă a modelelor agentice, ci sugerează că un agent generalist de programare nu este optim adaptat unei sarcini determinate, rezolvabile mai eficient printr-un apel unic structurat. Disocierea riguroasă a celor două cauze ar necesita un experiment dedicat care să varieze cadrul de execuție al agentului (*harness*-ul), menținând modelul fixat.

Inferența locală este o alternativă viabilă, iar criteriul de selecție este contraintuitiv. Un model MoE de 35 de miliarde de parametri, pe o placă de 12 GiB, egalează cel mai bun agent din familia Claude și depășește patru dintre cele șapte sisteme agentice. Factorul limitativ nu este dimensiunea fișierului, ci densitatea arhitecturii: modelul cu cel mai mare volum ocupat pe disc atinge o viteză de peste șase ori mai mare decât un model dens de dimensiune semnificativ mai mică. Pentru orice organizație care nu poate trimite date în afara perimetrului propriu, acesta este rezultatul cu cea mai directă consecință practică.

Capacitatea modelului nu ordonează rezultatele. În configurația agentică, Opus 5 se clasează sub Sonnet 5 și sub Haiku 4.5; pe BIRD, modelul „pro” obține un scor inferior modelului „flash”, la un cost de cinci ori mai ridicat. Ambele situații reflectă aceeași dinamică: modelul cu o capacitate computațională superioară direcționează resursele către prudență decizională excesivă și explorare redundantă, generând refuzuri nejustificate în detrimentul răspunsurilor exacte. Alegerea modelului pentru un mediu de producție nu poate fi fundamentată exclusiv pe poziția acestuia în ierarhia comercială a furnizorului.

Un set de evaluare saturat ocultează diferențele dintre sisteme, iar soluția nu constă în sporirea complexității sintactice a interogărilor. Cât timp toate modelele obțin 100 %, niciun model nu poate fi declarat superior altuia. Creșterea dificultății sintactice a întrebărilor proprii nu a schimbat această

situație; schimbarea autorului întrebărilor a schimbat-o imediat, coborând aceleași modele la 56–65 % și redând astfel măsurătorii puterea de a le ordona. Mai mult, dispersia de 61 de puncte între bazele BIRD, față de cele 8 puncte dintre modele, arată că *domeniul și particularitățile bazei de date* au un impact mai mare asupra performanței finale decât *alegerea modelului*.

Lacuna reală de capacitate este ambiguitatea. Este singura dimensiune măsurată care nu s-a saturat. Modelele detectează ambiguitatea explicită și o ratează sistematic pe cea latentă: la întrebări care par determinate, dar admit mai mulți numitori apărăbili, toate cele cinci modele cu apel unic au ales implicit o variantă fără a solicita clarificări. Accesul prin unelte inversează acest rezultat, ceea ce oferă un argument măsurat (singurul din lucrare) în favoarea accesului la date pentru clarificare, și justifică mecanismul de clarificare din sistemul propriu.

5.3 Direcții de dezvoltare

1. **Măsurarea conduitei RAG proprii față de setul curent de întrebări:** singura celulă absentă din capitolul 4 și, în consecință, prima prioritate. Ar permite testarea directă a ipotezei că recuperarea curatoriată se comportă diferit de explorarea agentică.
2. **Refacerea studiului de varianță** pe setul curent, pentru a stabili pragul sub care diferențele nu sunt semnificative.
3. **Punctarea ambiguității pe trei stări** (răspuns tăcut, răspuns cu ipoteză enunțată explicit, cerere de clarificare), ceea ce ar transforma singura dimensiune nesaturată într-o metrică utilizabilă.
4. **Extinderea configurației agentice la AdventureWorks**, pentru a stabili dacă penalizarea agentică crește sau scade cu dimensiunea schemei.
5. **O configurație hibridă:** un singur apel pentru generarea SQL, cu unelte disponibile *exclusiv* pentru rezolvarea ambiguității. Rezultatele din secțiunile 4.4 și 4.7 sugerează împreună că aceasta este configurația care combină avantajele fără costul lor.
6. **Măsurarea pe scheme de ordinul sutelor de tabele**, regimul în care Spider 2.0 [3] și BEAVER [4] arată că sistemele actuale coboară sub 20 % și asupra cărora prezenta lucrare nu formulează concluzii empirice.

Bibliografie

- [1] T. Yu, R. Zhang, K. Yang *et al.*, „Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”, în *Proceedings of EMNLP 2018*, Association for Computational Linguistics, Bruxelles, Belgia, 2018, pp. 3911–3921. <https://arxiv.org/abs/1809.08887>
- [2] J. Li, B. Hui, G. Qu *et al.*, „Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs”, în *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, New Orleans, SUA, 2023. <https://arxiv.org/abs/2305.03111>
- [3] F. Lei, J. Chen, Y. Ye *et al.*, „Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows”, în *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*, Singapore, 2025. <https://arxiv.org/abs/2411.07763>, consultat în august 2026.
- [4] P. B. Chen, D. Yang, W. Li *et al.*, „BEAVER: An Enterprise Benchmark for Text-to-SQL”, preprint arXiv, 2024. <https://arxiv.org/abs/2409.02038>, consultat în august 2026.
- [5] T. Jin, Y. Choi, Y. Zhu și D. Kang, „Text-to-SQL Benchmarks are Broken: An In-Depth Analysis of Annotation Errors”, în *Proceedings of the Conference on Innovative Data Systems Research (CIDR 2026)*, Santa Cruz, SUA, 2026. <https://www.vldb.org/cidrdb/papers/2026/p5-jin.pdf>, consultat în august 2026.
- [6] K. Maamari, F. Abubaker, D. Jaroslawicz și A. Mhedhbi, „The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models”, în *Proceedings of the NeurIPS 2024 Third Table Representation Learning Workshop*, Vancouver, Canada, 2024. <https://arxiv.org/abs/2408.07702>, consultat în august 2026.
- [7] X. Liu, S. Shen, B. Li, N. Tang și Y. Luo, „NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation”, în *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Toronto, Canada, 2025. <https://arxiv.org/abs/2503.11984>, consultat în august 2026.
- [8] Z. Yao, G. Sun, L. Borchmann *et al.* (Snowflake AI Research, University of Maryland, University of California San Diego), „Arctic-Text2SQL-R1: Simple Rewards, Strong Reasoning in Text-to-SQL”, preprint arXiv, 2025. <https://arxiv.org/abs/2505.20315>, consultat în august 2026.
- [9] Agentic AI Foundation (protocol inițiat de Anthropic în 2024, donat fundației în decembrie 2025), „Model Context Protocol — Specification (2025-06-18)”, documentație tehnică on-line. <https://modelcontextprotocol.io/specification/2025-06-18>, consultat în august 2026.
- [10] Z. Luo, X. Shi, X. Lin și J. Gao, „Evaluation Report on MCP Servers”, preprint arXiv, 2025. <https://arxiv.org/abs/2504.11094>, consultat în septembrie 2026.
- [11] D. Gao, H. Wang, Y. Li *et al.*, „Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation”, în *Proceedings of the VLDB Endowment*, vol. 17, nr. 5, 2024, pp. 1132–1145. <https://www.vldb.org/pvldb/vol17/p1132-gao.pdf>, consultat în septembrie 2026.
- [12] A. Bhaskar, T. Tomar, A. Sathe și S. Sarawagi, „Benchmarking and Improving Text-to-SQL Generation under Ambiguity”, în *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023, pp. 7053–7074. <https://arxiv.org/abs/2310.13659>, consultat în septembrie 2026.

- [13] M. Dong, N. A. Kumar, Y. Hu *et al.*, „PRACTIQ: A Practical Conversational Text-to-SQL Dataset with Ambiguous and Unanswerable Queries”, preprint arXiv, 2024. <https://arxiv.org/abs/2410.11076>, consultat în septembrie 2026.
- [14] BIRD-SQL, „Leaderboard”, clasament on-line actualizat continuu. <https://bird-bench.github.io/>, consultat în august 2026.
- [15] Anthropic, „Claude Code, agentic coding tool”, documentație de produs on-line. <https://claude.com/product/claude-code>, consultat în august 2026.
- [16] OpenAI, „Codex CLI — coding agent that runs in your terminal”, documentație de produs on-line. <https://developers.openai.com/codex/cli/>, consultat în august 2026.
- [17] PostgreSQL Global Development Group, „PostgreSQL — documentație oficială”. <https://www.postgresql.org/docs/>, consultat în august 2026.
- [18] SQLite Consortium, „SQLite: documentație oficială”. <https://www.sqlite.org/docs.html>, consultat în august 2026.
- [19] Linux Foundation, „Linux Foundation Announces the Formation of the Agentic AI Foundation (AAIF), Anchored by New Project Contributions Including Model Context Protocol (MCP), goose and AGENTS.md”, comunicat de presă, decembrie 2025. <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>, consultat în septembrie 2026.
- [20] A. Vaswani, N. Shazeer, N. Parmar *et al.*, „Attention Is All You Need”, în *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, SUA, 2017, pp. 5998–6008. <https://arxiv.org/abs/1706.03762>
- [21] BIRD-SQL, „bird_sql_dev_20251106: revizia setului de dezvoltare”, HuggingFace Datasets. https://huggingface.co/datasets/birdsql/bird_sql_dev_20251106, consultat în august 2026.
- [22] Microsoft, „AdventureWorks sample databases”; portare PostgreSQL: Norfolk-DataSci, *adventure-works-postgres*, revizia afcfd2df. <https://github.com/NorfolkDataSci/adventure-works-postgres>, consultat în august 2026.
- [23] G. Gerganov *et al.*, „llama.cpp — LLM inference in C/C++”, proiect open-source. <https://github.com/ggml-org/llama.cpp>, consultat în august 2026.
- [24] O. Moll, „pgserver, embedded PostgreSQL server for Python”, Python Package Index. <https://pypi.org/project/pgserver/>, consultat în august 2026.
- [25] Alibaba Cloud, „Qwen3.5 și Qwen3.6, carduri de model”, documentație de produs; fără raport tehnic publicat la data redactării. Fișierele GGUF efectiv măsurate provin de la doi cuantizatori terți, lmstudio-community și unsloth, identificați prin numele exact al depozitului HuggingFace în anexa A. Consultat în august 2026.

A. Modelele măsurate

Cuantizarea este parte a măsurătorii. De aceea fiecare model local este identificat prin fișierul exact rulat.

Tabela A.1: Modelele locale, cu fișierul GGUF măsurat.

Model	Fișier GGUF
Arctic-Text2SQL-R1-7B [8]	mradermacher/Arctic-Text2SQL-R1-7B-GGUF, Q4_K_M (4,4 GiB) și Q8_0 (7,5 GiB)
Qwen3.5-9B [25]	lmstudio-community/Qwen3.5-9B-GGUF, Q4_K_M (5,2 GiB)
Qwen3.6-27B (dens)	unsloth/Qwen3.6-27B-GGUF, UD-IQ3_XXS (11,2 GiB)
Qwen3.6-35B-A3B (MoE, activare parțială)	unsloth/Qwen3.6-35B-A3B-GGUF, UD-Q4_K_M (20,6 GiB)
Bonsai-27B	lmstudio-community/Bonsai-27B-GGUF, Q1_0 (3,5 GiB). Metadatele GGUF nu conțin legătură base_model: partajează arhitectura Qwen3.6-27B, dar nu este o cuantizare a acestuia

Modelele din cloud sunt produse comerciale fără lucrări publicate. Ele sunt citate prin identificator și prin tariful în vigoare la momentul rulării (prices_checked 2026-08-23), înregistrat în fiecare fișier de rezultate.

Tabela A.2: Modelele din cloud măsurate.

Furnizor	Modele	Folosite în
Google	gemini-2.5-flash, gemini-3-flash-preview, gemini-3.1-pro-preview, gemini-3.6-flash, gemini-3.7-flash	configurație A (secțiunea 4.3), BIRD (secțiunea 4.8)
Anthropic	claude-opus-5, claude-sonnet-5, claude-haiku-4-5, claude-fable-5	configurație B, claude-agent (secțiunea 4.4)
OpenAI	gpt-5.6-sol, gpt-5.6-terra, gpt-5.6-luna	configurație B, codex-agent (secțiunea 4.4)

B. Reproducerea rezultatelor

Fiecare cifră din capitolul 4 este citită din `benchmark/results/*.json` de scripturile care generează tabelele și figurile. Niciun număr nu este transcris manual.

```
# tabelele si figurile din capitolul 4
python docs/diploma/figures/make_all.py

# o rulare a bratului A
python benchmark/run.py --arm naive --model gemini-3.7-flash
python benchmark/run.py --arm naive --model gemini-3.7-flash \
    --dataset adventureworks

# BIRD-SQL dev: construirea setului din cele doua artefacte upstream,
# cu validarea fiecărei interogari de referinta, apoi rulara
python benchmark_data/bird/prepare.py
./benchmark/sweep_bird.sh          # 3 modele, in paralel
DATASET=bird_dev_dict ./benchmark/sweep_bird.sh gemini-3.7-flash

# bratul local: lista modelelor, pornirea unuia, apoi sweep-ul complet
./benchmark/serve_local.sh --list
./benchmark/serve_local.sh qwen36-35b-moe
./benchmark/sweep_local.sh
```

Listing B.1: Comenzile care regenerează măsurătorile și materialul grafic al lucrării.

Fiecare fișier de rezultate înregistrează configurația, modelul, amprenta setului de întrebări, SQL-ul per întrebare, consumul de tokeni per întrebare și latența, ceea ce permite repunctarea fără reapelarea modelelor.

Structura seturilor de întrebări

Setul `car_rental` conține 51 de întrebări (44 punctate, 7 ambigue), dintre care 6 formulate în limba română. Setul `adventureworks` conține 56 de întrebări (49 punctate, 7 ambigue), dintre care 5 în limba română. Distribuția pe categorii este dată în tabelul 3.3; amprentele criptografice ale celor două seturi sunt `86e194123bb3`, respectiv `5398f281a907`, iar cea a setului `BIRD-SQL dev` este `e700eb224701`.

Categoriile marcate ca adăugate în etapa a doua au fost introduse după ce măsurătorile inițiale au arătat că grupurile *simple*, *agregare*, *join* și *subinterogare* încetaseră să diferențieze sistemele din configurațiile A și B — motivația introducerii acestora este documentată în secțiunea 4.6.